

Active Wind Power Forecasting Through Machine Learning Approaches

A project report

to

Submitted in partial fulfillment of the requirements for the award

Of

BACHELOR OF TECHNOLOGY

IN

CSE - CYBER SECURITY

Submitted

by

K. SRI PUTHRA VASAVI 22A21A4628

V. PRAKASH 22A21A4650

B. GAGANASRI 22A21A4636

Under the guidance

of

Dr. T. Murali Mohan

M.Tech., Ph.D.

Professor & HOD

Department of CSE-CS



DEPARTMENT OF CSE - CYBER SECURITY

SWARNANDHRA COLLEGE OF ENGINEERING AND TECHNOLOGY

(AUTONOMOUS)

(Approved by AICTE, Affiliated to JNTU Kakinada, Accredited by NBA and NAAC)

Seetharampuram , Narsapur-534275, West Godavari (Dist.), AP.

2025 - 2026

DEPARTMENT OF CSE - CYBER SECURITY
SWARNANDHRA COLLEGE OF ENGINEERING AND TECHNOLOGY
(AUTONOMOUS)

(Approved by AICTE, Affiliated to JNTU Kakinada, Accredited by NBA and NAAC)
Seetharampuram, Narsapur-534275, West Godavari (Dist.), AP.



CERTIFICATE

This is to certify that the project report entitled “**Active Wind Power Forecasting Through Machine Learning Approaches**” is a bona fide work done by **K.SRI PUTHRA VASAVI - 22A21A4628, V.PRAKASH - 22A21A4650, B.GAGANA SRI - 22A21A4636**, submitted in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Department of **CSE-CYBER SECURITY**, during the academic year 2024 - 2026.

Project Guide

Dr. T. Murali Mohan, M. Tech.,
Ph. D., Professor & HOD
Department of CSE-CS

Head of the Department

Dr. T. Murali Mohan, M. Tech.,
Ph. D., Professor & HOD
Department of CSE-CS

External Examiner

DECLARATION

We certify that the project work presented in this report is original and has been carried out by us under the guidance of our supervisor. This work has not been submitted to any other University or institution for award of any degree or diploma. All the guidelines and standards prescribed by the college have been strictly followed while preparing and writing this project report.

PLACE : Seetharampuram

DATE :

NAME OF THE STUDENTS	REGD.NO	SIGNATURE
K.SRI PUTHRA VASAVI	22A21A4628	
V.PRAKASH	22A21A4650	
B.GAGANASRI	22A21A4636	

ACKNOWLEDGEMENT

We sincerely express my gratitude to the Management for providing all the necessary facilities, constant encouragement, and unwavering support throughout my course of study.

We are deeply indebted to **Dr. S. Suresh Kumar, M. Tech, Ph. D,PDF , Principal**, and **Dr. A. Gopi Chand , M. Tech, Ph. D , Vice Principal** for taking the time to share his knowledge, for his keen interest in my work, and for his invaluable guidance and kind advice. His constant encouragement and insightful suggestions have greatly contributed to the successful completion of my project.

We extend our heartfelt thanks to **Dr. T. Murali Mohan, M. Tech, Ph. D, Professor & HOD**, Department of CSE - CYBER SECURITY, for his valuable suggestions and insightful guidance in software analysis and design, as well as his continuous support in ensuring the successful completion of this project.

We are especially grateful to our project guide **Dr. T Murali Mohan., M. Tech, Ph.D**, Department of CSE - CYBER SECURITY, for his continuous support, patience, motivation, and enthusiasm. His immense knowledge and guidance were instrumental in the successful completion of this assignment.

We extend our sincere appreciation to **Ms. N. Jeevana Jyothi, M. Tech., (Phd)** Assistant Professor, Department of CSE- CYBER SECURITY, for her guidance and unwavering support as the project coordinator, which played a crucial role in the project's successful execution.

We heartfelt gratitude to Swarnandhra College of Engineering and Technology for providing us with the opportunity and resources to successfully undertake our project.Finally, we extend our gratitude to everyone who has directly or indirectly contributed to this endeavor.

PROJECT ASSOCIATES

K.SRI PUTHRA VASAVI	22A21A4628
V.PRAKASH	22A21A4650
B.GAGANA SRI	22A21A4636

CONTENTS

ABSTRACT.....	01
CHAPTER 1.....	03
INTRODUCTION.....	03
1. INTRODUCTION.....	04
CHAPTER 2.....	07
LITERATURE SURVEY.....	07
2. LITERATURE SURVEY.....	08
2.1. SUPPORT VECTOR MACHINE (SVM) FOR WIND POWER FORECASTING..	08
2.2 K – NEAREST NEIGHBOUR (KNN) FOR WIND POWER PREDICTION.....	09
2.3. ARTIFICIAL NEURAL NETWORKS (ANN) AND HYBRID MODELS.....	10
2.4. ADVANCED MODELS : BAYESIAN , GENETIC PROGRAMMING , AND RANDOM FOREST.....	11
CHAPTER 3.....	16
SYSTEM ANALYSIS.....	16
3. SYSTEM ANALYSIS.....	17
3.1 EXISTING SYSTEM.....	17
3.2 DISADVANTAGES.....	18
3.3 PROPOSED SYSTEM.....	19
3.4 ADVANTAGES.....	20
3.5 MODULES.....	21
CHAPTER 4.....	22
IMPLEMENTATION.....	22
4. IMPLEMENTATION.....	23
CHAPTER 5.....	30
SOFTWARE REQUIREMENT SPECIFICATION.....	30
5. SOFTWARE REQUIREMENT SPECIFICATION.....	31
5.1 REQUIREMENTS SPECIFICATION.....	31
CHAPTER 6.....	36
METHODOLOGY.....	36
6. METHODOLOGY.....	37
CHAPTER 7.....	45
SYSTEM DESIGN & UML DESIGN.....	45

7. SYSTEM DESIGN.....	45
7.1 SYSTEM ARCHITECTURE.....	46
7.1.1 DATA FLOW DIAGRAM.....	47
7.2 UML DIAGRAMS.....	48
7.2.1 USE CASE DIAGRAM.....	49
7.2.2 SEQUENCE DIAGRAM.....	49
7.3 ACTIVITY DIAGRAM.....	51
7.3.2 CLASS DIAGRAM.....	52
CHAPTER 8.....	53
SYSTEM TESTING.....	53
8. TESTING.....	54
CHAPTER 9.....	59
SAMPLE CODE.....	60
CHAPTER 10.....	65
SCREENSHOTS.....	65
10. SCREEN SHOTS.....	66
CHAPTER 11.....	74
CONCLUSION.....	74
11. CONCLUSION.....	75
REFERENCE.....	77

LIST OF TABLES

Table No	Table Name	Page No
5.1	Minimum Hardware Requirements	35
8.1	Test cases	59

LIST OF FIGURES

Figure no	Figure Name	Page No
4.1	Work Flow	27
4.2	Forecasting Dataset and Tools	28
4.3	Essential Steps in Data Pre-processing	29
4.4	Training and Testing Data Split	30
4.5	Regression Models	32
6.1	Analysis stage	42
6.2	Designing stage	43
6.3	Coding stage	45
6.4	Integration and Testing Stage	46
6.5	Installation	47
7.1.1	System Architecture	49
7.1.2	Data Flow	50
7.2.1	Use Case Diagram	52
7.2.2	Sequence Diagram	53
7.3.1	Activity Diagram	54
7.3.2	Class Diagram	55

9.1	Home Page	70
9.2	User Registration Page	71
9.3	Admin Login Form	72
9.4	Activate Users	73
9.5	User Login Form	74
9.6	View Datasets	75
9.7	ML Results	76
9.8	Prediction Page	76
9.8	Prediction Output	77

ABSTRACT

ABSTRACT

Wind power is a non-conventional, promising, and widely accessible renewable energy resource. Accurately predicting the useful wind power production from wind turbines is of paramount importance for efficient management of energy in power grids. This project aims to estimate and forecast active wind power generation using machine learning regression models. It evaluates models including **Linear Regression, Ridge Regression, Lasso Regression, Decision Tree Regression, Gradient Boosting Regression, and K-Nearest Neighbour Regression**. The data used consists of real-time wind turbine readings collected every 10 minutes.

Advanced data pre-processing techniques, such as imputation for missing values, outlier treatment, and dimensionality reduction using **Principal Component Analysis (PCA)**, were employed to enhance data quality and model performance. The system also incorporates correlation analysis to identify key contributing features such as Wind Speed, Rotor RPM, Generator Temperature, and Reactive Power.

Extensive model training and evaluation were performed using industry-standard metrics including **Mean Absolute Error (MAE), Mean Square Error (MSE), R-squared (R^2), and Mean Absolute Percentage Error (MAPE)**. Among all tested algorithms, the K-Nearest Neighbour Regression exhibited the best performance, demonstrating high accuracy and robustness for short-term wind power forecasting. The outcomes of this project can assist energy providers in optimizing power generation schedules, reducing energy imbalance, and improving overall grid stability by integrating intelligent forecasting systems.

CHAPTER 1

INTRODUCTION

1. INTRODUCTION

The growing concern over environmental degradation and climate change has significantly accelerated the global shift from conventional fossil fuels to renewable energy sources. Fossil fuels such as coal, oil, and natural gas are not only finite but also major contributors to greenhouse gas emissions, leading to global warming and environmental imbalance. As a result, countries around the world are investing heavily in sustainable and eco-friendly alternatives to meet their increasing energy demands. Among the various renewable energy sources available, wind energy has emerged as one of the most promising and rapidly expanding solutions due to its abundance, renewability, and minimal environmental impact.

Wind energy harnesses the natural power of wind through turbines to generate electricity without emitting harmful pollutants. Unlike traditional energy sources, wind power generation does not produce carbon dioxide or other greenhouse gases during operation, making it a crucial component in achieving global sustainability goals and reducing dependence on fossil fuels. Additionally, advancements in wind turbine technology and decreasing installation costs have made wind energy more economically viable, further encouraging its adoption across both developed and developing nations.

However, despite its numerous advantages, wind energy presents a major challenge—its inherent variability and unpredictability. Wind patterns are influenced by multiple environmental factors such as wind speed, wind direction, temperature, humidity, and atmospheric pressure. These factors change continuously and are difficult to predict with absolute accuracy. As a result, the amount of electricity generated by wind turbines fluctuates over time, making it challenging to maintain a stable balance between energy supply and demand. This intermittency can lead to inefficiencies in power systems, including overproduction, underutilization, or even grid instability.

To overcome these challenges, accurate wind power forecasting has become a critical requirement in modern energy systems. Forecasting enables energy providers and grid operators to estimate future wind power generation and plan accordingly. With reliable predictions, operators can optimize energy distribution, schedule backup power sources, and ensure efficient utilization of energy resources. This not only enhances grid stability but also minimizes operational costs and reduces energy wastage.

Traditional wind power forecasting methods primarily rely on statistical models and physical equations. While these approaches have been useful in the past, they often struggle to capture the complex and nonlinear relationships present in real-world wind data. Wind behavior is influenced by dynamic interactions between multiple variables, making it difficult for conventional models to provide highly accurate predictions. Consequently, there is a growing need for more advanced and intelligent forecasting techniques that can handle such complexities effectively.

In recent years, Machine Learning (ML) has emerged as a powerful tool for solving complex prediction problems across various domains, including renewable energy. Machine learning algorithms are capable of learning patterns from large datasets and making accurate predictions without being explicitly programmed. By analysing historical data, ML models can identify hidden relationships between input variables and output power generation, leading to improved forecasting accuracy.

This project focuses on the application of machine learning techniques for active wind power forecasting. The system utilizes a dataset collected from wind turbines at regular intervals of 10 minutes, capturing detailed information about operational and environmental conditions. Key input features include wind speed, rotor speed, generator speed, blade pitch angles, and other atmospheric parameters. These features play a crucial role in determining the performance and power output of wind turbines.

To develop an effective forecasting system, multiple regression-based machine learning models are implemented and evaluated. These models include Linear Regression, Ridge Regression, Lasso Regression, Decision Tree Regression, Gradient Boosting Regression, and K-Nearest Neighbours (KNN). Each of these algorithms has its own strengths and characteristics in handling different types of data patterns. For instance, Linear Regression provides a simple baseline model, while Decision Trees and Gradient Boosting are capable of capturing nonlinear relationships. Regularization techniques like Ridge and Lasso help in reducing overfitting and improving model generalization.

The performance of these models is evaluated using appropriate metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), and R-squared (R^2) score. By comparing

these metrics, the best-performing model is selected for accurate wind power prediction. This systematic evaluation ensures that the chosen model delivers reliable and consistent results under varying conditions.

Another important aspect of this project is its scalability and adaptability. The developed system is designed to be flexible enough to integrate with real-time monitoring platforms and smart grid systems. With further enhancements, the model can be deployed in live environments where it continuously receives real-time data and updates predictions accordingly. This capability is particularly useful for modern smart grids, where dynamic decision-making is essential for efficient energy management.

Furthermore, accurate wind power forecasting contributes to better energy planning and resource management. It allows energy providers to reduce reliance on backup power sources, optimize storage systems, and improve overall grid efficiency. In addition, it supports the integration of renewable energy into existing power systems without compromising reliability or performance. This ultimately leads to reduced operational costs and a more sustainable energy infrastructure.

In conclusion, the integration of machine learning techniques in wind power forecasting represents a significant advancement in renewable energy management. By addressing the limitations of traditional methods and leveraging data-driven approaches, this project aims to provide a robust and efficient solution for predicting wind energy output. The proposed system not only enhances forecasting accuracy but also supports the broader goal of achieving a cleaner, greener, and more sustainable energy future.

A comparative study for the fake banknote recognition task between a custom model and several models obtained by transfer learning, in terms of accuracy and inference times is given. The rest of the paper is organized as follows. Section presents the background of Convolutional Neural Networks and transfer learning. Section shows the proposed system of image acquisition and the used dataset. Section explains the proposed methodology for selecting the freezing point in the design by transfer learning. Section presents the proposed custom model. Section shows the results of the research. Finally, Section summarizes the work.

CHAPTER 2

LITERATURE SURVEY

2. LITERATURE SURVEY

The accurate forecasting of wind power output is essential for effective energy management, especially as the integration of renewable energy into modern power systems increases. Over the years, a variety of machine learning (ML) and artificial intelligence (AI) techniques have been explored to address the challenges associated with wind power prediction. These methods aim to overcome the limitations of traditional statistical and physics-based models, which often struggle with the nonlinear and dynamic behavior of wind patterns.

In recent literature, the **Support Vector Machine (SVM)** has gained popularity for its ability to model complex patterns in wind data. Studies have demonstrated its effectiveness in wind power forecasting when used with appropriately selected meteorological inputs.

K-Nearest Neighbors (KNN) is another widely used algorithm in wind power prediction. It has shown promising results due to its simplicity and capacity to perform well with noisy datasets. One study applied KNN to historical meteorological data and successfully forecasted wind power with high accuracy.

Artificial Neural Networks (ANNs) and their variants such as **Feedforward Neural Networks (FNNs)** and **Recurrent Neural Networks (RNNs)** have also been applied to time series prediction of wind power. In particular, hybrid approaches that combine **Empirical Mode Decomposition (EMD)** with ANNs have been explored to decompose wind speed data into intrinsic modes before applying forecasting models, resulting in more refined and accurate predictions.

Bayesian models have been implemented to enhance probabilistic forecasting by incorporating uncertainty and regional wind data. These models are particularly useful for short-term and ultra-transient predictions. However, their complexity may limit their practicality in some real-time applications.

Another study investigated the use of **Genetic Programming (GP)** to build ensembles of neural networks for wind power forecasting. This method provided a robust framework for combining multiple weak learners to enhance predictive accuracy.

Random Forests, both with and without random feature selection, have also been compared

against other regression models. Results showed that Random Forests and SVMs achieved comparable performance levels, highlighting the importance of model selection based on dataset characteristics and forecasting requirements.

From these studies, it is evident that machine learning models provide a flexible, data-driven approach to forecasting wind power. They outperform traditional models in many scenarios, particularly when dealing with large volumes of real-time data. However, selecting the right model and preprocessing techniques remains critical for achieving high accuracy and reliability.

In this project, multiple machine learning algorithms such as Linear Regression, Ridge Regression, Lasso Regression, Decision Tree, Gradient Boosting, and K-Nearest Neighbors (KNN) are implemented and evaluated for wind power forecasting. Each model is trained using real-time wind turbine data collected at regular intervals

Before applying these algorithms, data preprocessing techniques such as handling missing values, normalization, and feature selection are performed to improve model performance. Important features like wind speed, rotor speed, temperature, and pressure are considered as input variables for prediction.

Among all the implemented models, K-Nearest Neighbors (KNN) demonstrated better performance in terms of accuracy and robustness for short-term forecasting. This is because KNN effectively captures local patterns in the data and performs well even with noisy datasets.

The performance of all models is evaluated using standard metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), and R-squared (R^2). These evaluation metrics help in selecting the best model for accurate wind power prediction

Overall, this project highlights the importance of selecting suitable machine learning algorithms and proper data preprocessing techniques to achieve reliable and efficient wind power forecasting in real-world applications..

2.1. SUPPORT VECTOR MACHINE (SVM) FOR WIND POWER FORECASTING

Support Vector Machine (SVM) is a powerful supervised learning algorithm used for both classification and regression problems. In wind power forecasting, SVM is primarily used for regression tasks, where it predicts continuous output values such as wind power generation based on input features. The main objective of SVM is to find an optimal hyperplane that minimizes prediction error while maintaining a balance between model complexity and generalization.

SVM is particularly effective in handling nonlinear relationships by using kernel functions such as linear, polynomial, and radial basis function (RBF). These kernels allow the algorithm to transform input data into higher-dimensional space, where complex patterns can be more easily identified. In the context of wind power forecasting, SVM models utilize meteorological parameters such as wind speed, temperature, humidity, and atmospheric pressure as input features to predict power output.

Several research studies have demonstrated the effectiveness of SVM in short-term wind power forecasting. The model performs well even with limited datasets and provides high accuracy when properly tuned. One of the key strengths of SVM is its ability to avoid overfitting, especially in high-dimensional data scenarios.

However, SVM also has certain limitations. The performance of the model depends heavily on the selection of kernel functions and hyperparameters such as the regularization parameter and kernel coefficients. Improper tuning can lead to reduced accuracy. Additionally, SVM can be computationally intensive when dealing with large datasets, making it less suitable for real-time large-scale applications without optimization.

Despite these challenges, SVM remains one of the most reliable and widely used techniques in wind power forecasting due to its robustness and ability to model complex nonlinear relationships effectively.

2.2 K-NEAREST NEIGHBOURS (KNN) FOR WIND POWER PREDICTION

K-Nearest Neighbours (KNN) is a simple yet effective machine learning algorithm widely used for regression and classification problems. Unlike parametric models, KNN does not assume any predefined functional relationship between input and output variables. Instead, it relies on the similarity between data points to make predictions.

In wind power forecasting, KNN works by identifying the ‘k’ closest data points (neighbours) to a given input based on a distance metric such as Euclidean distance. The predicted output is calculated as the average of these neighbouring values. This approach makes KNN particularly useful for capturing local patterns and variations in wind data.

One of the key strengths of KNN is its simplicity and ease of implementation. It does not require a training phase, making it a lazy learning algorithm. This allows the model to adapt quickly to new data without retraining. Additionally, KNN performs well in scenarios where the dataset contains noise or irregular patterns, as it considers multiple nearby points rather than relying on a single prediction.

KNN is also highly flexible, as it can be used for both linear and nonlinear data without requiring complex transformations. It is particularly effective when the dataset is small to medium in size and when the relationship between variables is not well defined.

However, KNN has several limitations. One of the main drawbacks is its high computational cost during prediction, as it requires calculating distances between the input and all training data points. This makes it less suitable for large-scale datasets or real-time applications. Additionally, the performance of KNN depends heavily on the choice of ‘k’ value. A small value of ‘k’ may lead to overfitting, while a large value may smooth out important patterns.

Another challenge is the sensitivity of KNN to irrelevant features and scaling issues. Proper data preprocessing, including normalization and feature selection, is essential to achieve optimal performance. Despite these challenges, KNN remains a valuable technique in wind power forecasting due to its simplicity, adaptability, and effectiveness in handling noisy data.

2.3. ARTIFICIAL NEURAL NETWORKS (ANN) AND HYBRID MODELS

Artificial Neural Networks (ANNs) are among the most advanced and widely used machine learning techniques for wind power forecasting. Inspired by the human brain, ANNs consist of interconnected neurons organized in multiple layers, including input, hidden, and output layers. These networks are capable of learning complex nonlinear relationships between input variables and output values.

In wind power forecasting, ANNs are used to model the relationship between meteorological parameters and power generation. The network is trained using historical data, where it adjusts its weights and biases to minimize prediction error. Once trained, the model can accurately predict future wind power output based on new input data.

Different types of neural networks have been used in this domain. Feedforward Neural Networks (FNNs) are the most basic type, where data flows in one direction from input to output. Recurrent Neural Networks (RNNs) are designed for sequential data and can capture temporal dependencies, making them ideal for time series forecasting. Advanced models such as Long Short-Term Memory (LSTM) networks further improve performance by addressing issues such as vanishing gradients.

Hybrid models have been developed to enhance the performance of ANNs. One popular approach is the combination of Empirical Mode Decomposition (EMD) with neural networks. In this method, wind speed data is decomposed into multiple components, which are then processed separately by neural networks. This helps in reducing noise and improving prediction accuracy.

ANNs offer several advantages, including high accuracy, ability to model complex relationships, and adaptability to large datasets. However, they also require significant computational resources and large amounts of training data. Additionally, the training process can be time-consuming and may require careful tuning of parameters.

2.4. ADVANCED MODELS: BAYESIAN, GENETIC PROGRAMMING, AND RANDOM FOREST

With the increasing complexity of wind power forecasting problems, traditional machine learning models alone are sometimes insufficient to achieve the desired level of accuracy and reliability. As a result, researchers have explored advanced machine learning and artificial intelligence techniques that can better handle uncertainty, nonlinear relationships, and large-scale datasets. Among these advanced approaches, Bayesian models, Genetic Programming (GP), and Random Forest have gained significant attention due to their effectiveness in improving forecasting performance.

These techniques not only enhance prediction accuracy but also provide additional benefits such as uncertainty estimation, robustness, and adaptability. This section presents a detailed discussion of these advanced models, their methodologies, advantages, limitations, and applications in wind power forecasting.

Bayesian models are based on Bayes' Theorem, which provides a mathematical framework for updating probabilities based on new evidence. Unlike traditional deterministic models, Bayesian approaches focus on probabilistic forecasting, where predictions are expressed in terms of probability distributions rather than single point estimates.

In wind power forecasting, Bayesian models are particularly useful because they can account for uncertainty in both input variables and model predictions. Wind energy systems are highly sensitive to changes in weather conditions, and it is often difficult to predict exact values. Bayesian models address this issue by providing confidence intervals along with predictions, allowing operators to make more informed decisions.

The working principle of Bayesian models involves combining prior knowledge with observed data to obtain posterior distributions. The prior distribution represents initial beliefs about the system, while the likelihood function represents the probability of observing the data given certain parameters. By applying Bayes' Theorem, the model updates its beliefs and generates posterior distributions that reflect both prior knowledge and new data.

In practical applications, Bayesian models use meteorological data such as wind speed, direction, temperature, and pressure to predict wind power output. These models are particularly effective in short-term and ultra-short-term forecasting, where uncertainty plays a critical role. For example, Bayesian networks can model the dependencies between different variables and provide probabilistic predictions.

One of the major advantages of Bayesian models is their ability to quantify uncertainty. This is extremely important in power system operations, where decisions must be made under uncertain conditions. By providing probability distributions, these models help operators assess risks and plan accordingly.

Another advantage is their flexibility in incorporating prior knowledge. This is useful in scenarios where historical data is limited or when expert knowledge is available. Bayesian models can also be updated dynamically as new data becomes available, making them suitable for real-time applications.

However, Bayesian models also have certain limitations. One of the main challenges is their computational complexity, especially when dealing with large datasets and complex models. The process of calculating posterior distributions can be time-consuming and may require advanced techniques such as Markov Chain Monte Carlo (MCMC) methods.

Additionally, selecting appropriate prior distributions can be difficult and may influence the results. If the prior assumptions are incorrect, the model may produce biased predictions. Despite these challenges, Bayesian models remain a powerful tool for wind power forecasting, particularly in applications where uncertainty needs to be explicitly considered. Genetic Programming (GP) is an evolutionary algorithm inspired by the principles of natural selection and genetics. It is used to automatically generate mathematical models or programs that best fit a given dataset. In wind power forecasting, GP is used to develop predictive models by evolving solutions over multiple generations.

The working process of GP begins with an initial population of randomly generated models. Each model is evaluated based on a fitness function, which measures how well it predicts wind power output. The best-performing models are selected and undergo genetic operations

such as crossover (combining parts of two models) and mutation (random changes). This process is repeated over several generations, resulting in increasingly accurate models.

One of the key advantages of GP is its ability to discover complex relationships without requiring predefined model structures. Unlike traditional machine learning algorithms, GP does not rely on a fixed mathematical equation. Instead, it evolves models dynamically, allowing it to capture nonlinear and intricate patterns in wind data.

In wind power forecasting, GP has been used to create ensemble models by combining multiple weak learners into a strong predictive model. These ensembles improve accuracy and robustness by reducing the impact of individual model errors. GP can also be integrated with other techniques such as neural networks and fuzzy systems to further enhance performance.

Another important advantage of GP is its interpretability. The models generated by GP are often represented as mathematical expressions or decision trees, making them easier to understand compared to black-box models like neural networks.

However, GP also has several limitations. The computational cost of GP is relatively high, as it requires evaluating a large number of models over multiple generations. This can make it time-consuming and resource-intensive, especially for large datasets.

Additionally, GP may suffer from issues such as overfitting, where the model becomes too complex and fits the training data too closely. Proper regularization and validation techniques are required to address this issue.

Despite these challenges, GP is a powerful technique for wind power forecasting, particularly in scenarios where complex relationships need to be modelled and high accuracy is required.

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve prediction accuracy and reduce overfitting. It is one of the most widely used machine learning techniques in wind power forecasting due to its robustness, simplicity, and effectiveness.

The basic idea behind Random Forest is to create a large number of decision trees using different subsets of the training data. Each tree is trained independently, and the final prediction is obtained by averaging the outputs of all trees (in regression tasks). This approach reduces the variance of predictions and improves overall accuracy.

In wind power forecasting, Random Forest models use input features such as wind speed, direction, temperature, and turbine parameters to predict power output. The algorithm automatically selects important features and ignores irrelevant ones, making it highly efficient.

One of the major advantages of Random Forest is its ability to handle large datasets with high dimensionality. It can process both numerical and categorical data and is less sensitive to noise and outliers. Additionally, Random Forest provides feature importance measures, which help in identifying the most influential variables in the prediction process.

Another key advantage is its resistance to overfitting. Since the model combines multiple trees, it reduces the risk of overfitting that is common in individual decision trees. This makes Random Forest highly reliable for real-world applications.

Random Forest is also relatively easy to implement and requires minimal parameter tuning compared to other machine learning algorithms. This makes it a practical choice for both researchers and industry applications.

However, Random Forest also has some limitations. The model can become complex and difficult to interpret when a large number of trees are used. Additionally, it may require significant computational resources for training and prediction, especially with very large datasets.

Despite these challenges, Random Forest has been widely used in wind power forecasting and has demonstrated excellent performance in various studies. It is particularly effective in handling nonlinear relationships and large-scale data.

When comparing Bayesian models, Genetic Programming, and Random Forest, it is clear that each technique has its own strengths and is suitable for different applications.

- Bayesian models are ideal for probabilistic forecasting and uncertainty analysis.
- Genetic Programming is effective in discovering complex relationships and creating flexible models.
- Random Forest provides high accuracy, robustness, and ease of implementation.

In many cases, hybrid approaches that combine these techniques can achieve even better performance. For example, combining Random Forest with Bayesian methods can provide both accurate predictions and uncertainty estimation.

The choice of model depends on various factors such as dataset size, computational resources, prediction requirements, and application context. For real-time applications, simpler and faster models may be preferred, while for research purposes, more complex models can be used to achieve higher accuracy.

CHAPTER 3

SYSTEM ANALYSIS

3. SYSTEM ANALYSIS

System analysis is an important phase in the development of any project, as it helps in understanding the current system, identifying its limitations, and proposing an improved solution. In the context of wind power forecasting, system analysis focuses on evaluating existing forecasting methods, identifying their drawbacks, and designing a more efficient and accurate system using modern machine learning techniques.

Wind energy forecasting plays a crucial role in energy management systems. However, traditional approaches often fail to provide accurate predictions due to the dynamic and nonlinear nature of wind patterns. Therefore, it becomes necessary to analyze the existing system and develop a more advanced and reliable forecasting model.

This section provides a detailed discussion of the existing system, its disadvantages, the proposed system, its advantages, and the various modules involved in the system.

3.1 EXISTING SYSTEM

The existing system for wind power forecasting primarily relies on traditional statistical and physical models. These methods include time series analysis techniques such as Autoregressive (AR), Moving Average (MA), and ARIMA models, as well as Numerical Weather Prediction (NWP) models.

Statistical models use historical data to identify patterns and trends in wind speed and power generation. These models assume that future values can be predicted based on past observations. While they are simple and easy to implement, they often fail to capture complex nonlinear relationships present in wind data.

Physical models, on the other hand, use mathematical equations based on atmospheric physics to predict wind behavior. These models consider factors such as air pressure, temperature, and terrain conditions. Although they provide a theoretical understanding of wind patterns, they require extensive computational resources and may not always produce accurate results due to uncertainties in weather conditions.

In many cases, these traditional methods are used independently, without integrating real-time data or advanced analytics. As a result, the predictions are often less accurate and may not be suitable for dynamic energy systems.

Additionally, existing systems lack scalability and flexibility. They are not designed to handle large volumes of data or adapt to changing environmental conditions. This limits their effectiveness in modern smart grid applications, where real-time decision-making is essential. Overall, the existing system provides a basic framework for wind power forecasting but is limited in its ability to handle complex and dynamic data.

3.2 DISADVANTAGES

The existing wind power forecasting systems suffer from several limitations, which affect their accuracy, efficiency, and reliability.

One of the major disadvantages is the inability to handle nonlinear relationships. Wind patterns are highly complex and influenced by multiple interacting factors. Traditional models often assume linear relationships, which leads to inaccurate predictions.

Another limitation is the lack of adaptability. Existing systems are not capable of learning from new data or adjusting their models dynamically. This makes them less effective in real-time applications where conditions change frequently.

High computational complexity is also a concern, especially in physical models such as NWP. These models require significant processing power and time, making them unsuitable for quick decision-making.

Data dependency is another issue. Statistical models rely heavily on historical data and may not perform well when there are sudden changes in weather conditions. They also struggle with missing or noisy data.

Furthermore, existing systems often lack user-friendly interfaces and visualization tools. This makes it difficult for users to interpret the results and make informed decisions.

Scalability is another major drawback. Traditional systems are not designed to handle large datasets or integrate with modern technologies such as IoT and smart grids.

In summary, the limitations of existing systems highlight the need for a more advanced and efficient forecasting solution.

3.3 PROPOSED SYSTEM

The proposed system aims to overcome the limitations of existing methods by utilizing advanced machine learning techniques for wind power forecasting. It is designed to provide accurate, reliable, and real-time predictions of wind power output.

The system uses historical and real-time data collected from wind turbines, including parameters such as wind speed, direction, temperature, pressure, rotor speed, and blade pitch angle. This data is pre-processed to remove noise and inconsistencies, ensuring high-quality input for model training.

Multiple machine learning regression models are implemented, including Linear Regression, Ridge Regression, Lasso Regression, Decision Tree, Gradient Boosting, and K-Nearest Neighbours. These models are trained and evaluated to determine the best-performing algorithm for prediction.

The proposed system is capable of handling nonlinear relationships and complex patterns in the data. It continuously learns from new data, improving its accuracy over time. This makes it highly suitable for real-time applications.

The system also includes data visualization features that present predictions in the form of graphs and charts. This helps users easily understand trends and make informed decisions.

Additionally, the system is scalable and can be integrated with existing energy management systems. It supports real-time monitoring and forecasting, making it ideal for smart grid applications.

3.4 ADVANTAGES

The proposed system offers several advantages over traditional forecasting methods.

One of the key advantages is improved accuracy. Machine learning models are capable of capturing complex relationships in the data, resulting in more accurate predictions.

The system is also highly adaptable. It can learn from new data and update its models dynamically, ensuring consistent performance even under changing conditions.

Another advantage is reduced computational complexity compared to physical models. Machine learning models can process large datasets efficiently, making them suitable for real-time applications.

The system provides better data handling capabilities, including preprocessing, normalization, and feature selection. This improves the overall quality of predictions.

User-friendly visualization is another important benefit. The system presents results in an easy-to-understand format, enabling users to make quick decisions.

Scalability is also a major advantage. The system can handle large datasets and can be extended to include advanced models and additional features. Furthermore, the system supports integration with modern technologies such as IoT and smart grids, making it future-ready.

3.5 MODULES

The proposed system is divided into several modules, each responsible for a specific function. This modular approach improves system organization, maintainability, and scalability.

1. DATA COLLECTION MODULE

This module is responsible for collecting data from various sources, including wind turbines and meteorological sensors. It gathers parameters such as wind speed, direction, temperature, and pressure.

2. DATA PREPROCESSING MODULE

This module cleans and prepares the data for analysis. It handles missing values, removes noise, normalizes data, and performs feature selection to improve model performance.

3. MODEL TRAINING MODULE

In this module, different machine learning models are trained using the processed data. The models learn patterns and relationships between input variables and output power generation.

4. MODEL EVALUATION MODULE

This module evaluates the performance of trained models using metrics such as MAE, MSE, and R^2 score. It helps in selecting the best-performing model.

5. PREDICTION MODULE

The selected model is used to predict future wind power output based on new input data. This module provides accurate and real-time forecasts.

6. EVALUATION MODULE

This module presents the prediction results in the form of graphs, charts, and dashboards. It helps users understand trends and make informed decisions.

7. USER INTERFACE MODULE

This module provides an interactive interface for users to input data and view results. It ensures ease of use and accessibility.

CHAPTER 4

IMPLEMENTATION

4. IMPLEMENTATION

Introduction:

In this section, we describe the step-by-step implementation of the Wind Power Forecasting System using Machine Learning techniques. The implementation follows a structured workflow that includes data collection, preprocessing, model selection, training, evaluation, and prediction.

The goal of this implementation is to build a robust system that can accurately forecast wind power output based on historical and real-time turbine data. The system is developed using Python and various machine learning libraries, ensuring flexibility, scalability, and ease of integration.

Python Libraries that would be need to achieve the task:

- 1.Numpy
- 2.Pandas
- 3.Sci-kit Learn
- 4.Matplotlib

Understanding The Machine Learning Workflow:

We can define the machine learning workflow in 3 stages.

- 1.Gathering data
- 2.Data pre-processing
- 3.Researching the model that will be best for the type of data
- 4.Training and testing the model
- 5.Evaluation

Python Libraries Used

The following libraries are used to implement the system:

1. **NumPy** – for numerical computations
2. **Pandas** – for data manipulation and analysis
3. **Scikit-learn** – for machine learning models
4. **Matplotlib** – for data visualization

These libraries provide efficient tools for handling data, building models, and visualizing results.

Three Types of Data

1. Numeric e.g. income, age
2. Categorical e.g. gender, nationality
3. Ordinal e.g., low/medium/high

How can data pre-processing be performed?

These are some of the basic pre — processing techniques that can be used to convert raw data.

1. Conversion of data: As we know that Machine Learning models can only handle numeric features, hence categorical and ordinal data must be somehow converted into numeric features.
2. Ignoring the missing values: Whenever we encounter missing data in the data set then we can remove the row or column of data depending on our need. This method is known to be efficient but it shouldn't be performed if there are a lot of missing values in the dataset.
3. Filling the missing values: Whenever we encounter missing data in the data set then we can fill the missing data manually, most commonly the mean, median or highest frequency value is used.
4. Machine learning: If we have some missing data then we can predict what data shall be present at the empty position by using the existing data.
5. Outliers' detection: There are some error data that might be present in our data set that deviates drastically from other observations in a data set. [Example: human weight = 800 Kg; due to mistyping of extra 0]

Researching the model that will be best for the type of data.

Our main goal is to train the best performing model possible, using the pre- processed data.

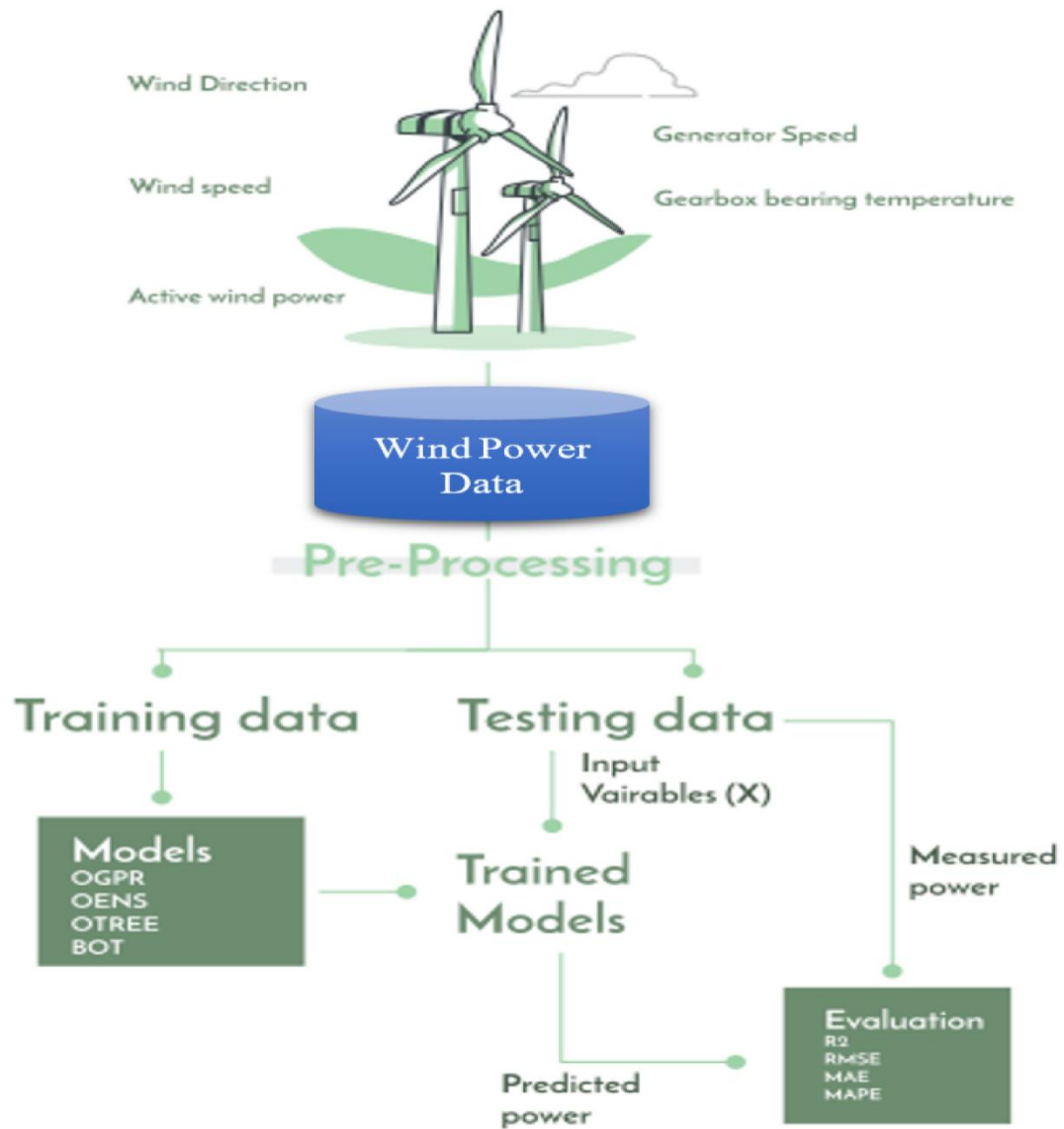


Fig no. 4.1 Work Flow

Understanding the Workflow

The implementation of the machine learning model is divided into the following steps:

- Gathering Data
- Data Pre-processing
- Model Selection
- Training and Testing
- Evaluation
- Hyper parameter Tuning
- Prediction

A machine learning model is a program that learns patterns from data and uses those patterns to make predictions. In this project, the model learns the relationship between input features such as wind speed and output power generation.

The accuracy of the model depends on the quality of the data. If poor or noisy data is used, the model will produce inaccurate predictions.

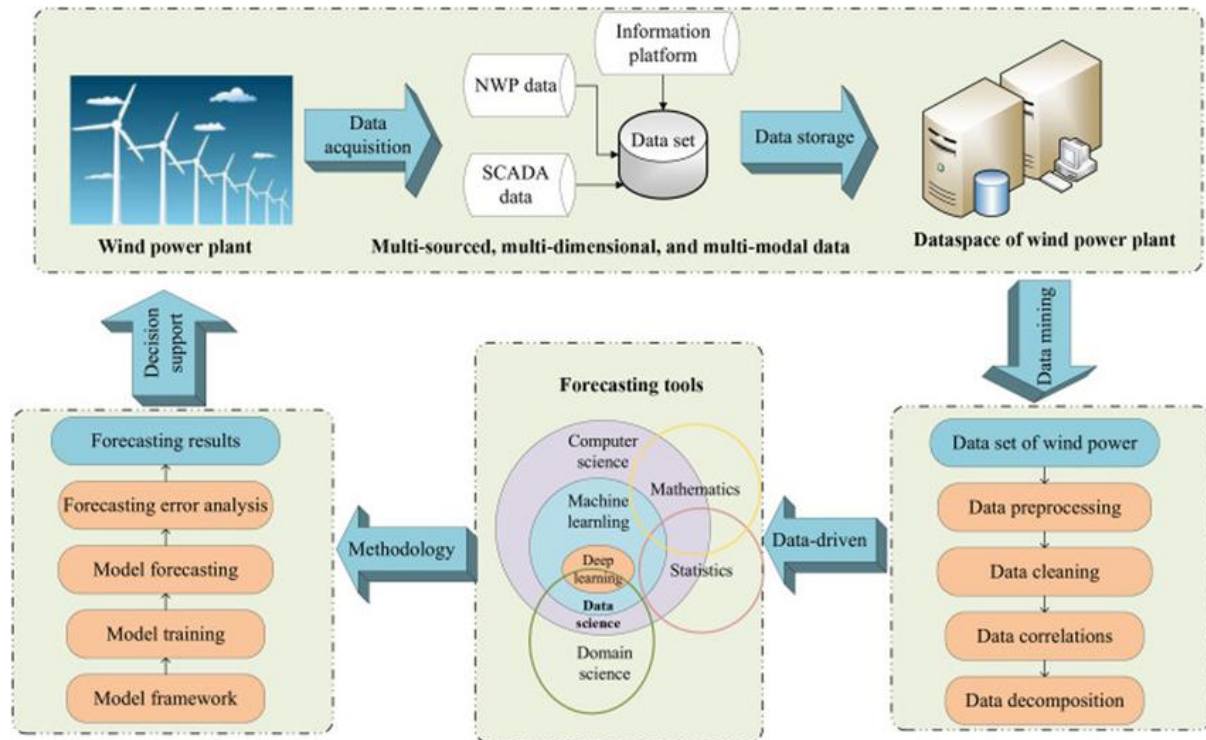


Fig no. 4.2 Forecasting Dataset and Tools

The dataset used in this project is collected from wind turbines and contains measurements recorded at regular intervals (e.g., every 10 minutes). The dataset includes:

- Wind Speed
- Wind Direction
- Temperature
- Pressure
- Rotor Speed
- Blade Pitch Angle
- Generated Power Output

The dataset can be obtained from:

- Kaggle datasets
- Wind turbine sensors
- Public repositories

Since real-world data is often noisy and incomplete, it cannot be used directly. Therefore, preprocessing is required.



Fig no. 4.3 Essential Steps in Data Preprocessing

Data preprocessing is one of the most important steps in the implementation. It ensures that the data is clean, consistent, and suitable for model training.

What is Data Pre-processing?

It is the process of converting raw data into a clean and structured format for analysis.

Why is it needed?

Real-world data may contain:

- Missing values
- Noisy data (outliers)
- Inconsistent data

Steps in Data Pre-processing

1. Handling Missing Values

- Replace with mean/median
- Remove rows if necessary

2. Removing Noise and Outliers

- Detect abnormal values
- Remove or correct them

3. Feature Selection

- Select important features like wind speed, rotor speed

4. Normalization

- Scale values for better performance

5. Data Conversion

- Convert categorical values into numeric format

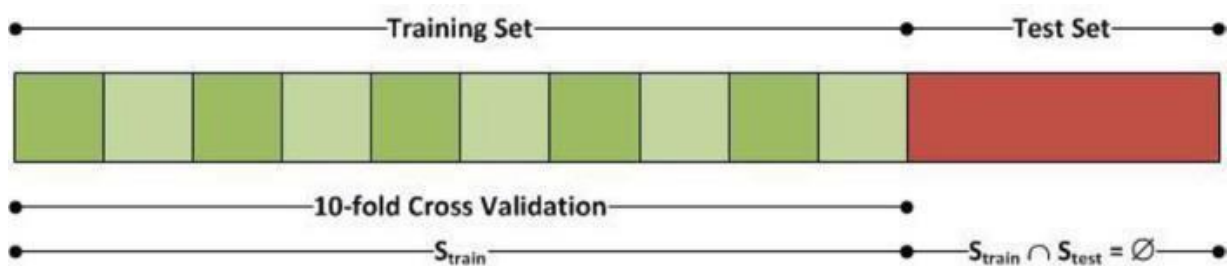


Fig no. 4.4 Training and Testing Data Split

Types of Data

The dataset includes:

- **Numeric Data** → wind speed, power output
- **Categorical Data** → turbine status (if present)
- **Ordinal Data** → low/medium/high wind levels

The next step is selecting appropriate machine learning models. Since this is a regression problem (predicting continuous values), the following models are used:

Regression Models Used

- Linear Regression
- Ridge Regression
- Lasso Regression
- Decision Tree Regression
- Gradient Boosting Regression
- K-Nearest Neighbors (KNN)

LINEAR REGRESSION :

Linear Regression is a supervised learning algorithm used for predicting continuous values. It establishes a linear relationship between input features (such as wind speed, temperature) and output (wind power). The model tries to fit a straight line that best represents the data. It is simple, fast, and works well when the relationship is linear.

RIDGE REGRESSION :

Ridge Regression is an improved version of Linear Regression that adds a regularization term to reduce overfitting. It is useful when the dataset contains multicollinearity (high correlation between features). By adding a penalty to large coefficients, it improves model stability and performance.

LASSO REGRESSION :

Lasso Regression is another type of regularized regression that not only reduces overfitting but also performs feature selection. It shrinks some coefficients to zero, which helps in removing less important features. This makes the model simpler and more efficient.

DECISION TREE REGRESSION :

Decision Tree Regression is a non-linear model that splits the data into smaller subsets based on feature values. It creates a tree-like structure of decisions, where each node represents a condition. It is easy to understand and can capture complex relationships in data.

GRADIENT BOOSTING REGRESSION :

Gradient Boosting is an ensemble learning technique that builds multiple models sequentially. Each new model tries to correct the errors of the previous model. It improves accuracy and handles complex data patterns effectively, but it may require more computation time.

K-NEAREST NEIGHBORS (KNN) :

KNN is a simple and effective algorithm that predicts output based on the nearest data points. It calculates the distance between data points and selects the closest 'k' neighbors.

The final prediction is made by averaging their values. It works well for small datasets and provides good accuracy.

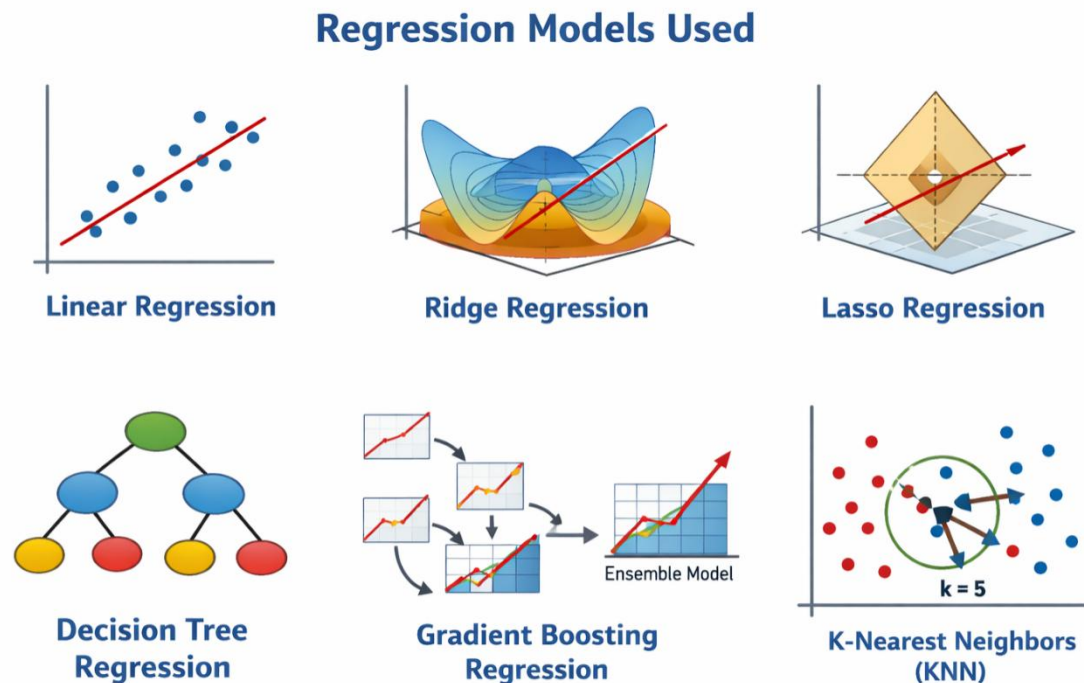


Fig no. 4.5 Regression Models

These models are chosen because they can handle both linear and nonlinear relationships.

Training and Testing the Model

The dataset is divided into three parts:

- Training Data – used to train the model
- Validation Data – used for tuning
- Testing Data – used to evaluate performance

Data Split Example

- 70% Training
- 15% Validation
- 15% Testing

The model is trained using training data and then tested on unseen data to check its accuracy.

CHAPTER 5

SOFTWARE REQUIREMENT SPECIFICATION

5. SOFTWARE REQUIREMENT SPECIFICATION

5.1 REQUIREMENTS SPECIFICATION

Requirement analysis is a critical phase in system development that identifies and defines the resources needed for the successful implementation of the project. It ensures that the system is designed within the constraints of available hardware and software while meeting performance and scalability requirements.

In the context of the wind power forecasting system, requirement analysis focuses on specifying the hardware infrastructure and software tools necessary for data processing, model training, evaluation, and deployment. A well-defined requirement analysis helps in ensuring smooth execution, optimal performance, and future scalability of the system.

5.1 HARDWARE REQUIREMENTS:

The hardware requirements define the physical components necessary to develop and run the wind power forecasting system efficiently. Since the project involves data processing and machine learning computations, the system must have sufficient processing power, memory, and storage capacity.

Processor

The processor plays a vital role in executing machine learning algorithms and handling data processing tasks. A system equipped with an Intel Core i5/i7 processor or an equivalent AMD processor is recommended. These processors provide multiple cores and threads, enabling parallel processing and faster execution of tasks such as data preprocessing, model training, and evaluation.

For basic machine learning models such as Linear Regression or Decision Trees, a mid-range processor is sufficient. However, for advanced models or large datasets, a higher-end processor improves performance significantly by reducing computation time.

RAM (Memory)

Memory is another critical component, especially when dealing with large datasets. A minimum of 8 GB RAM is required to run the system efficiently. However, 16 GB RAM is recommended for better performance, particularly when handling large datasets or running

multiple processes simultaneously.

Machine learning operations such as data loading, preprocessing, and model training consume a significant amount of memory. Insufficient RAM can lead to slow performance or system crashes. Therefore, having adequate memory ensures smooth execution and faster processing.

Storage

Storage is required to store datasets, model files, and output results. A minimum of 500 GB HDD or 256 GB SSD is recommended. While HDD provides larger storage capacity at a lower cost, SSD offers faster data access speeds, which improves overall system performance.

Using an SSD is highly beneficial for machine learning tasks, as it reduces the time required to read and write data. This is particularly useful when working with large datasets or performing multiple iterations during model training.

GPU (Optional)

A Graphics Processing Unit (GPU) is optional but highly beneficial for advanced machine learning and deep learning models. GPUs such as NVIDIA with CUDA support can significantly accelerate computations by performing parallel processing.

In this project, since traditional machine learning models are used, a GPU is not mandatory. However, if the system is extended to include deep learning models such as Neural Networks or LSTM, a GPU becomes highly advantageous.

System Architecture

A 64-bit system architecture is required to support modern operating systems and software tools. It allows the system to utilize more memory and process large datasets efficiently. Most modern systems are built on 64-bit architecture, making it a standard requirement.

Internet Connectivity

Internet connectivity is essential for downloading datasets, installing libraries, and accessing online resources. It is also required for integrating the system with cloud platforms or deploying the model for real-time applications.

In addition, internet access enables updates to software tools and libraries, ensuring that the system remains up-to-date with the latest features and security improvements.

Minimum Hardware Requirements

Overall, the hardware requirements for the wind power forecasting system are moderate and easily achievable with standard computing systems. The system is designed to run efficiently without requiring expensive or specialized hardware, making it accessible for academic and small-scale industrial applications.

Component	Specification
Processor	Intel Core i5/i7 or AMD equivalent
RAM	8 GB minimum (16 GB recommended)
Storage	500 GB HDD or 256 GB SSD
GPU (Optional)	NVIDIA GPU with CUDA support (for deep learning models)
System Architecture	64-bit
Internet Connectivity	Required for dataset download & model deployment

5.2 SOFTWARE REQUIREMENTS

Software requirements define the tools, programming languages, libraries, and frameworks needed to develop and implement the system. The proposed system relies on widely used and open-source software technologies, making it cost-effective and easy to maintain.

Operating System

The system is compatible with multiple operating systems, including Windows 10/11, Ubuntu 20.04 or higher, and macOS. This cross-platform compatibility ensures flexibility and allows users to choose their preferred environment.

Linux-based systems such as Ubuntu are often preferred for machine learning tasks due to their stability and performance. However, Windows and mac OS also provide sufficient support for development and execution.

Programming Language

Python 3.8 or higher is used as the primary programming language for this project. Python is widely in the field of data science and machine learning due to its simplicity, readability, and extensive library support.

Python enables rapid development and provides built-in functions for data manipulation, analysis, and visualization. Its large community support also ensures easy troubleshooting and access to learning resources.

IDE/Editor

Integrated Development Environments (IDEs) such as Jupyter Notebook, Visual Studio Code, and PyCharm are used for coding and development.

- **Jupyter Notebook** is ideal for data analysis and visualization, as it allows interactive execution of code.
- **VS Code** provides a lightweight and flexible coding environment.
- **PyCharm** offers advanced features for debugging and project management.

These tools improve productivity and make it easier to develop and test machine learning models.

Libraries and Frameworks

Several Python libraries are used in this project:

- **NumPy** – for numerical computations
- **Pandas** – for data manipulation
- **Scikit-learn** – for implementing machine learning models
- **Matplotlib & Seaborn** – for data visualization
- **XG Boost** – for advanced gradient boosting models

These libraries provide efficient and optimized functions for handling data and building predictive models.

Data Imputation

Handling missing data is an important part of preprocessing. The Iterative Imputer from Scikit-learn (based on the MICE technique) is used for imputing missing values.

This technique estimates missing values by modelling each feature as a function of other

features, resulting in more accurate and reliable data.

Model Evaluation

To evaluate model performance, various metrics are used:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- R-squared (R^2) Score
- Mean Absolute Percentage Error (MAPE)

These metrics help in comparing different models and selecting the best-performing one.

Visualization Tools

Visualization is an important aspect of the system, as it helps users understand data trends and prediction results. Tools such as Matplotlib and Seaborn are used to create graphs, charts, and plots.

These visualizations provide insights into model performance and help in decision-making.

Dataset Source

The dataset used in this project is obtained from Kaggle, specifically the Wind Turbine SCADA dataset. This dataset contains real-world data collected from wind turbines, making it suitable for training machine learning models.

Version Control

Version control tools such as Git are optionally used to manage the project code. Git helps in tracking changes, collaborating with others, and maintaining different versions of the project.

Summary of Software Requirements

The software requirements for the system are based on open-source technologies, making the project cost-effective and easy to implement. The tools used are widely supported and provide all the necessary functionalities for building a robust machine learning system.

Software requirements define the tools, programming languages, libraries, and frameworks needed to develop and implement the system. The proposed system relies on widely used and open-source software technologies, making it cost-effective and easy to maintain.

CHAPTER 6

METHODOLOGY

6. METHODOLOGY

SDLC (SOFTWARE DEVELOPMENT LIFE CYCLE) – UMBRELLA MODEL

Software Development Life Cycle (SDLC) is a systematic process used for developing high-quality software in a structured and efficient manner. The Umbrella Model of SDLC emphasizes continuous activities such as project management, quality assurance, documentation, and risk management throughout all phases of development.

In this project, the SDLC Umbrella Model is used to design and develop the Wind Power Forecasting System using machine learning techniques. Each phase of the SDLC is carefully followed to ensure that the system is reliable, scalable, and efficient.

Requirements Gathering Stage:

The requirements gathering stage is the initial and most important phase of the SDLC. In this stage, the objectives and functionalities of the wind power forecasting system are clearly identified.

The main goal of the system is to accurately predict wind power output using machine learning models. Based on this goal, the following requirements are identified:

- Ability to collect and process wind turbine data
- Support for multiple input parameters such as wind speed, temperature, and pressure
- Implementation of machine learning models for prediction
- Visualization of prediction results using graphs and charts
- User-friendly interface for easy interaction

Each requirement is documented with a unique identifier, description, and expected output. These requirements are compiled into a **Requirements Document**, which serves as a reference throughout the project.

A **Requirements Traceability Matrix (RTM)** is also created to map each requirement to specific project goals. This ensures that all requirements are addressed during development and helps in tracking progress.

Additionally, feasibility analysis is conducted to evaluate technical, economical, and social aspects of the project. Team responsibilities are defined, and modules are assigned accordingly.

The outputs of this stage include:

- Requirements Document
- Requirements Traceability Matrix (RTM)
- Initial Project Plan

Analysis Stage:

The analysis stage focuses on understanding the system requirements in detail and planning the overall structure of the project.

In this project, the analysis involves studying wind power generation and identifying the key factors that influence it. These include wind speed, direction, rotor speed, and environmental conditions.

The system is analysed to determine:

- Type of problem (Regression problem)
- Suitable machine learning algorithms
- Data requirements and sources
- Input-output relationships

A detailed study of existing systems is also conducted to identify their limitations. Based on this analysis, the proposed system is designed to overcome these challenges using advanced machine learning techniques.

The project plan is refined, including:

- Timeline for each phase
- Resource allocation

- Risk management strategies

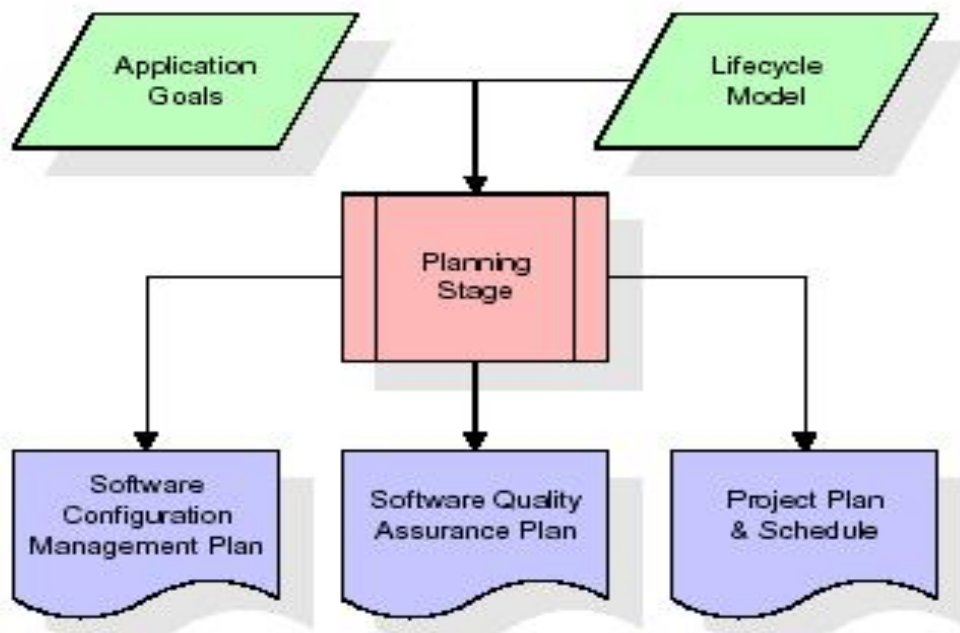


Fig no. 6.1 Analysis stage

The most critical section of the project plan is a listing of high-level product requirements, also referred to as goals. All of the software product requirements to be developed during the requirements definition stage flow from one or more of these goals.

The minimum information for each goal consists of a title and textual description, although additional information and references to external documents may be included.

The outputs of the project planning stage are the configuration management plan, the quality assurance plan, and the project plan and schedule, with a detailed listing of scheduled activities for the upcoming Requirements stage, and high-level estimates of effort for the out stages.

Designing Stage:

The design stage translates the requirements into a detailed system architecture. It provides a blueprint for the development phase.

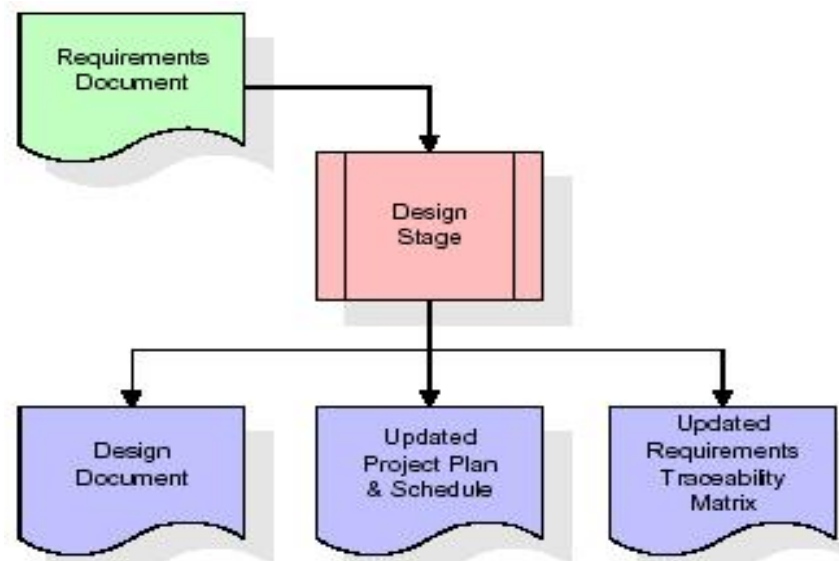


Fig no. 6.2 Designing stage

The system design includes:

1. System Architecture Design

- Input Layer (Data Collection)
- Processing Layer (Data Preprocessing & Model Training)
- Output Layer (Prediction & Visualization)

2. Data Flow Design

- Data is collected from datasets
- Preprocessing is applied
- Models are trained and evaluated
- Predictions are generated

3. Module Design

- Data Collection Module

- Data Pre-processing Module
- Model Training Module
- Evaluation Module
- Prediction Module
- Visualization Module

4. Database Design

- Dataset structure (features and target variable)
- Storage format (CSV/SQLite)

5. Interface Design

- Graphical representation of results
- User interaction screens

Design tools such as flowcharts, diagrams, and pseudo-code are used to represent system functionality clearly.

The outputs of this stage include:

- Design Document
- Updated RTM
- System Architecture Diagrams

Development (Coding) Stage:

In this stage, the actual implementation of the system is carried out based on the design specifications.

The system is developed using Python and various libraries such as:

- NumPy

- Pandas
- Scikit-learn
- Matplotlib

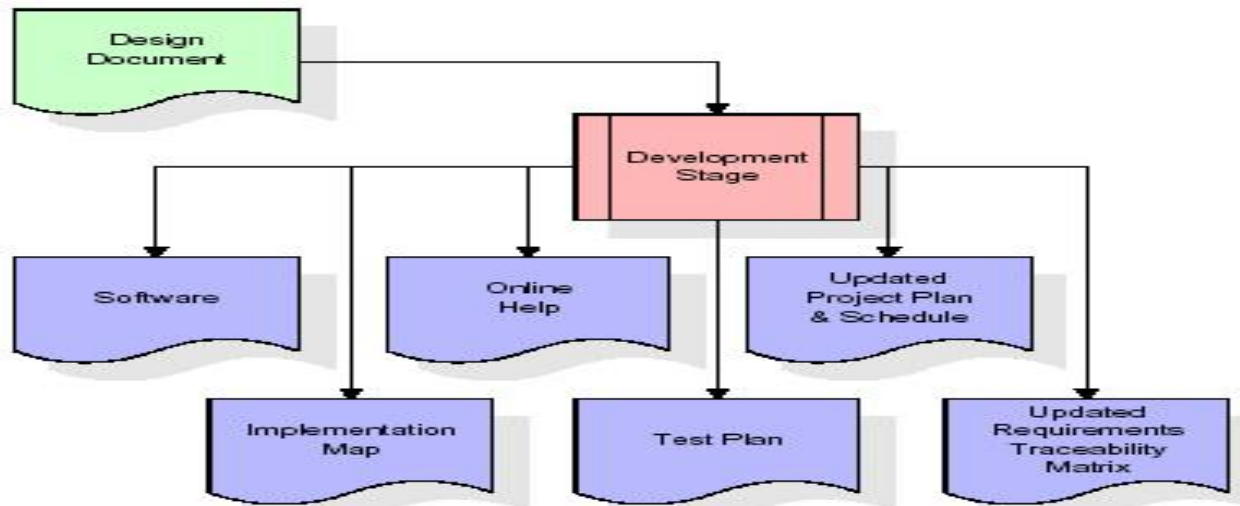


Fig no. 6.3 Coding stage

1. Importing required libraries
2. Loading the dataset
3. Performing data preprocessing
4. Splitting data into training and testing sets
5. Training multiple machine learning models
6. Evaluating model performance
7. Selecting the best model
8. Generating predictions

Each module is developed separately and then integrated into the system. Proper coding standards and documentation are maintained throughout this stage.

The outputs of this stage include:

- Source Code
- Trained Models
- Test Models

Integration & Test Stage:

During the integration and test stage, the software artifacts, online help, and test data are migrated from the development environment to a separate test environment. At this point, all test cases are run to verify the correctness and completeness of the software. Successful execution of the test suite confirms a robust and complete migration capability. During this stage, reference data is finalized for production use and production users are identified and linked to their appropriate roles. The final reference data (or links to reference data source files) and production user list are compiled into the Production Initiation Plan.

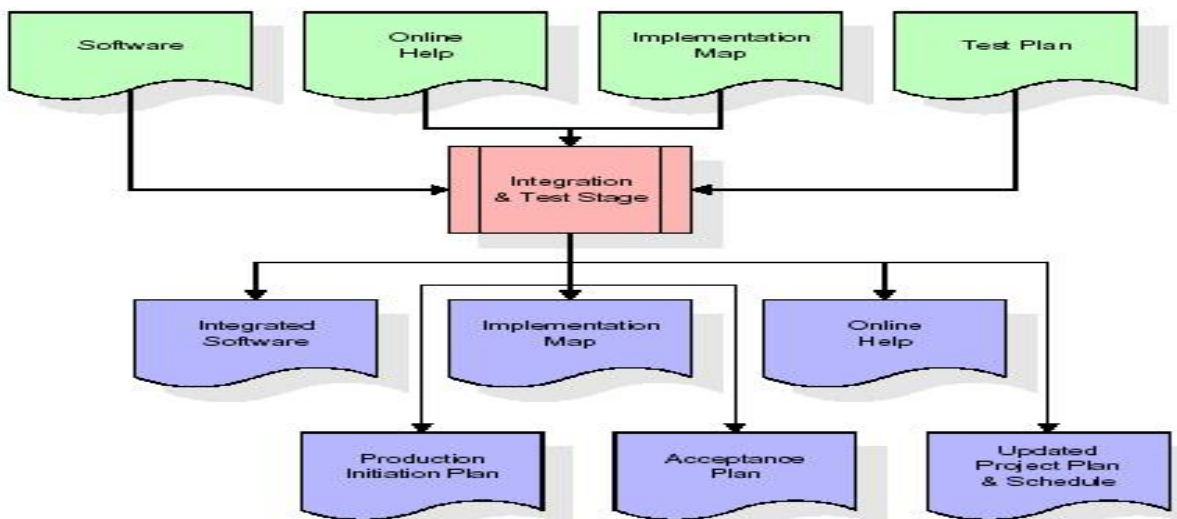


Fig no. 6.4 Integration and Testing Stage

Installation & Acceptance Test:

During the installation and acceptance stage, the software artifacts, online help, and initial production data are loaded onto the production server. At this point, all test cases are run to verify the correctness and completeness of the software. Successful execution of the test suite is a prerequisite to acceptance of the software by the customer.

After customer personnel have verified that the initial production data load is correct and the test suite has been executed with satisfactory results, the customer formally accepts the delivery of the software.

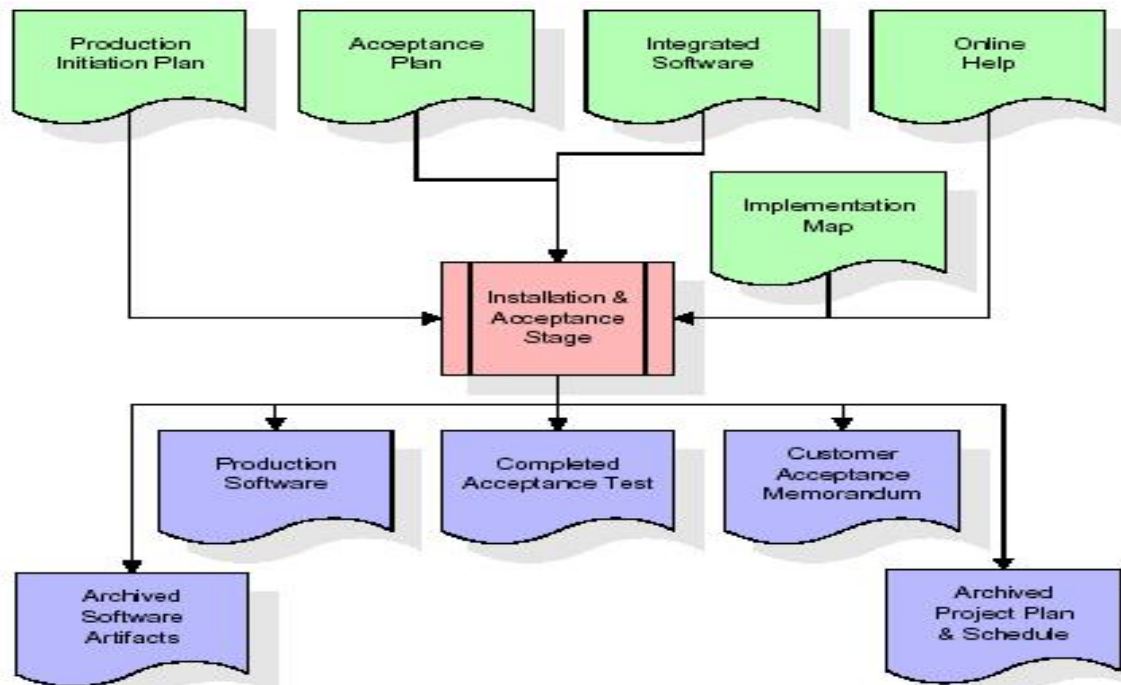


Fig no. 6.5 Installation

Maintenance:

Outer rectangle represents maintenance of a project, Maintenance team will start with requirement study, understanding of documentation later employees will be assigned work and they will undergo training on that particular assigned category.

CHAPTER 7

SYSTEM DESIGN & UML DESIGN

7. SYSTEM DESIGN

7.1 SYSTEM ARCHITECTURE

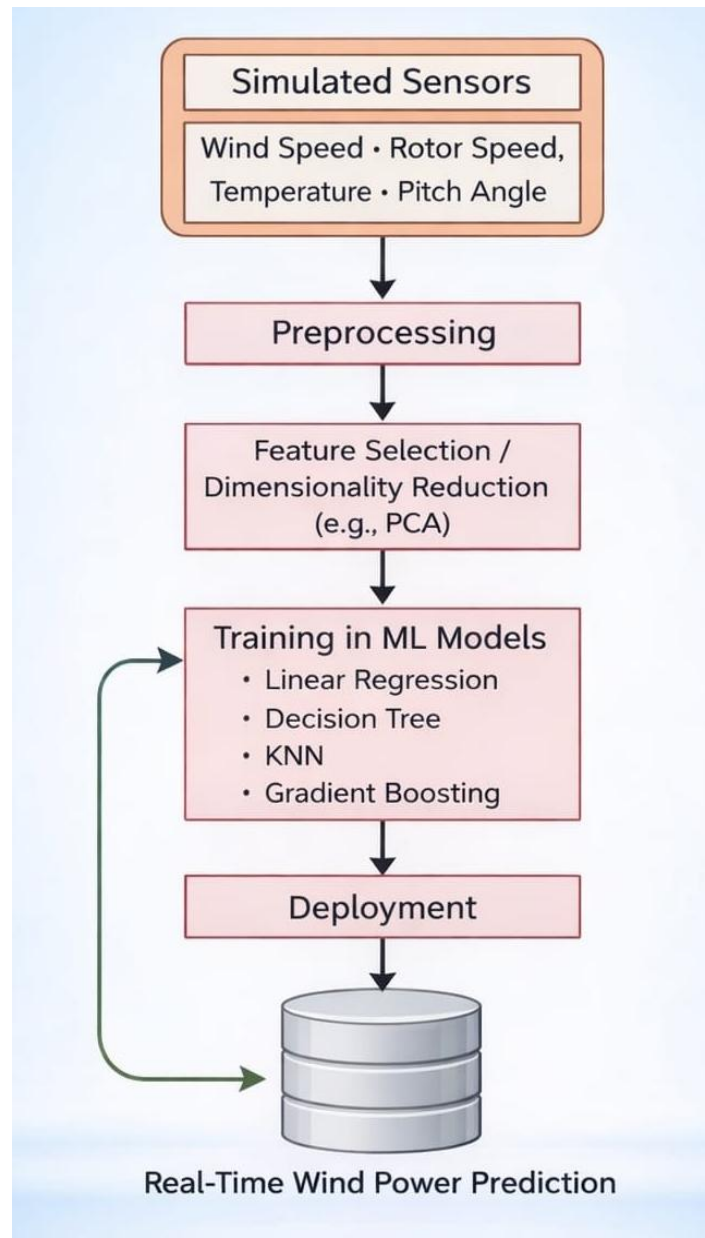


Fig.no. 7.1.1 Architecture diagram

DATA FLOW DIAGRAM:

1. The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.
2. The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system.
3. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output.
4. DFD is also known as bubble chart. A DFD may be used to represent a system at any level of abstraction. DFD may be partitioned into levels that represent increasing information flow and functional detail.

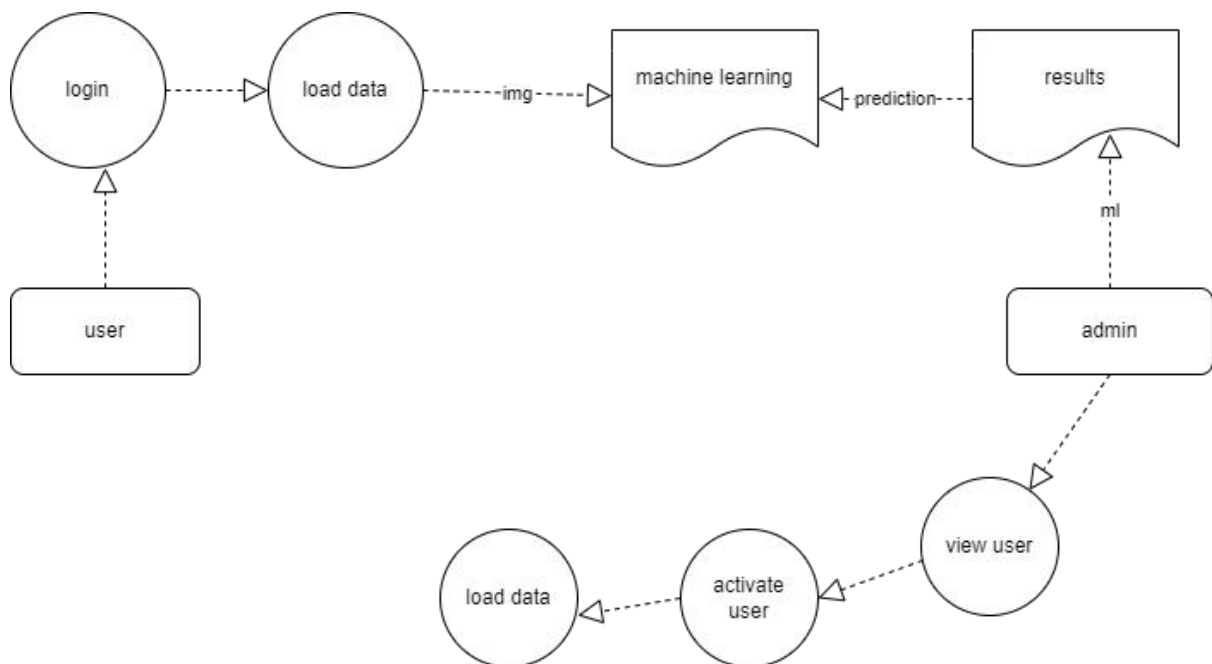


Fig.no. 7.1.2 Data Flow

7.2 UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.

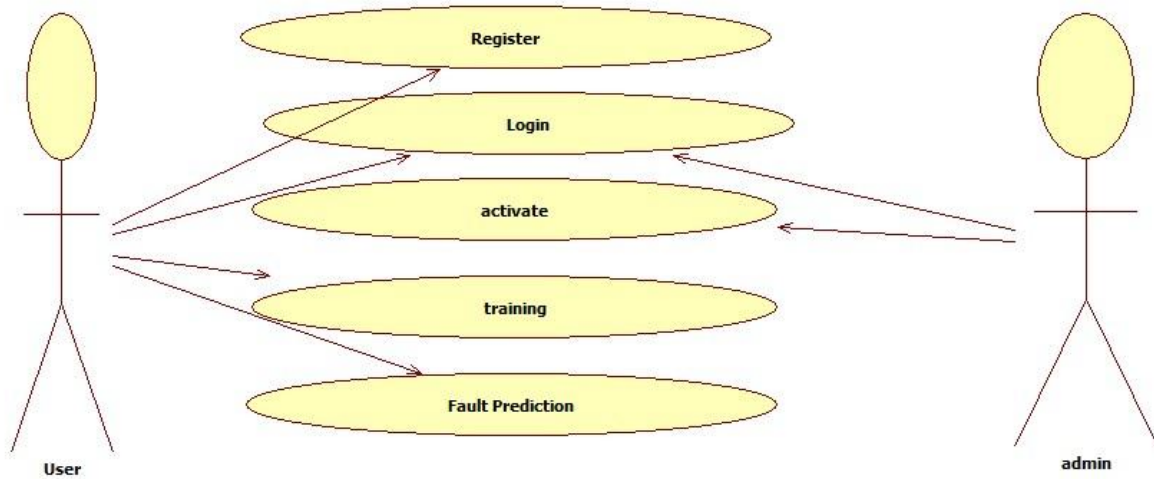


Fig no. 7.2.1 Use Case Diagram

7.2.2 SEQUENCE DIAGRAM

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

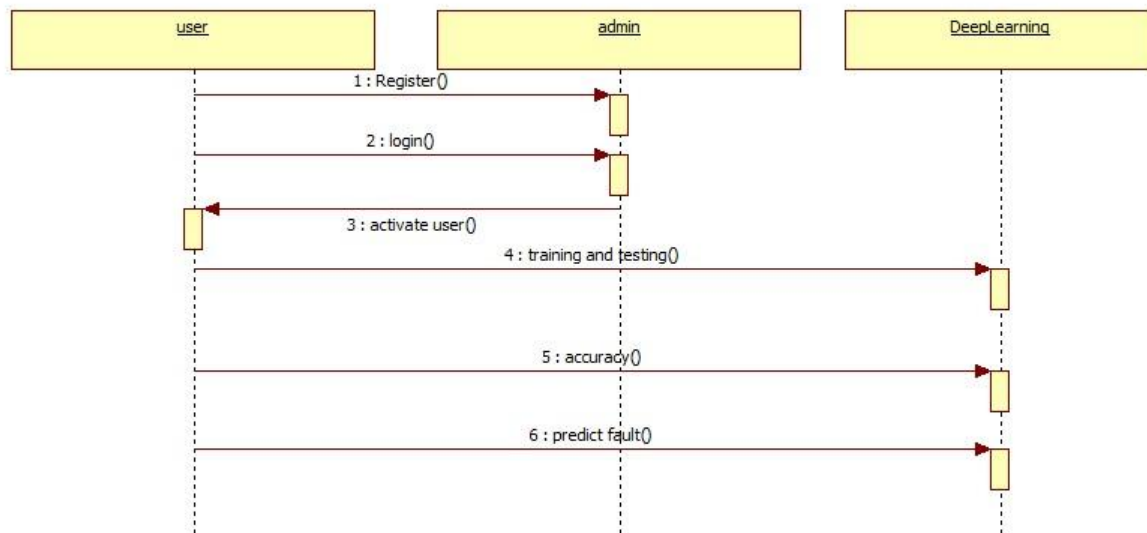


Fig no. 7.2.2 Sequence diagram

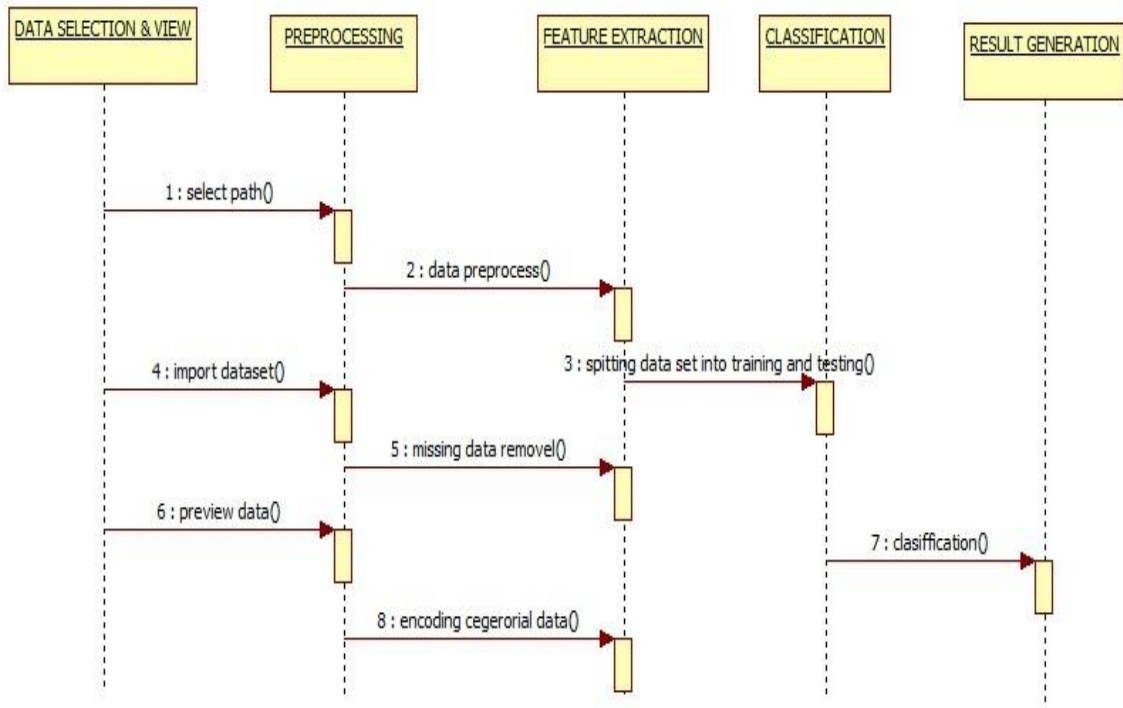


Fig no. 7.2.2 Sequence diagram

7.3 ACTIVITY DIAGRAM

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

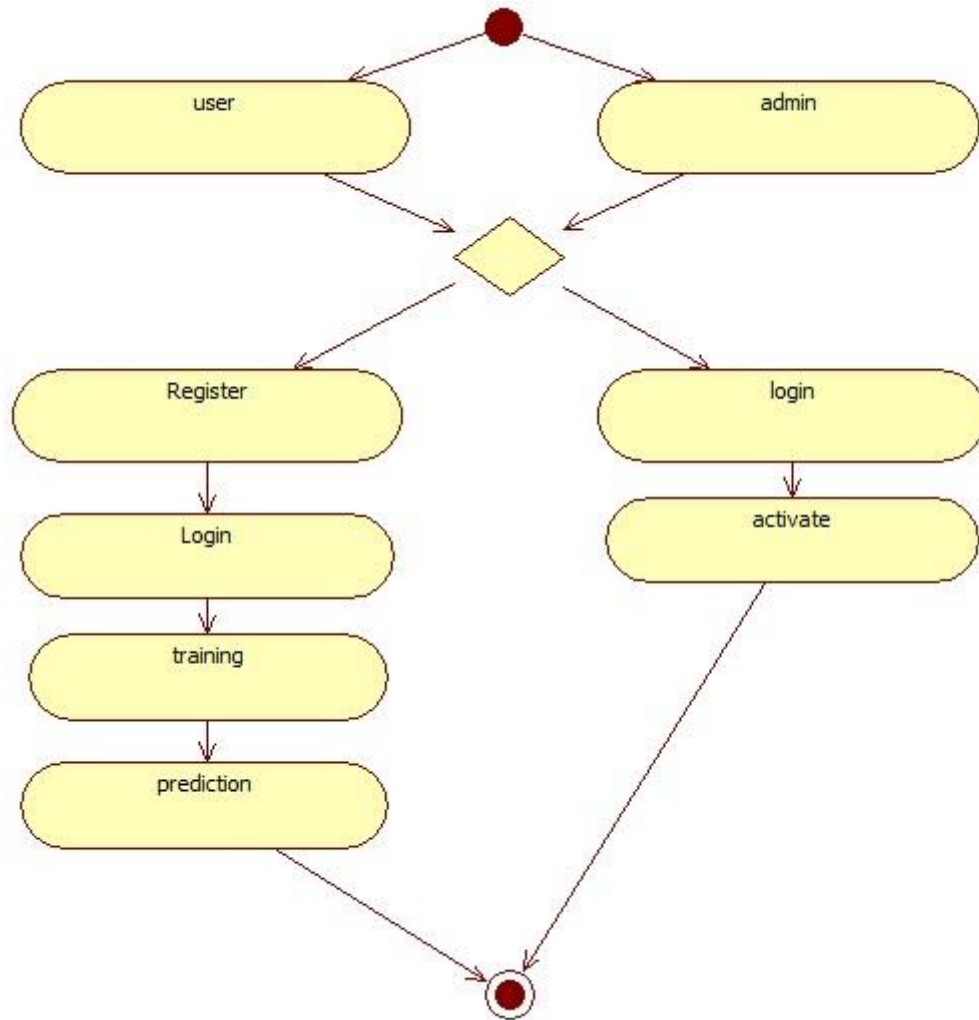


Fig no. 7.3.1 Activity Diagram

CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

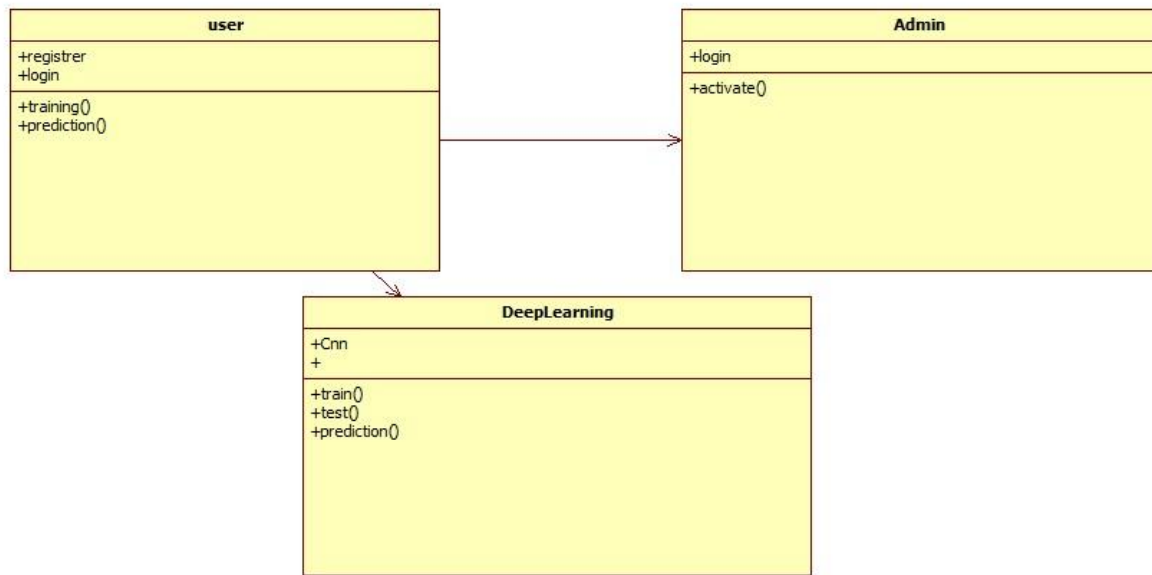


Fig no.7.3.2 Class Diagram

CHAPTER 8

SYSTEM TESTING

8. TESTING

The purpose of testing in this project is to ensure that the wind power forecasting system accurately processes historical and real-time turbine data, predicts power output correctly, and responds appropriately to user inputs. Testing ensures that the machine learning models are integrated correctly, the web interface works seamlessly, and all functionalities meet user and system requirements.

Testing helps validate both the backend logic (data preprocessing, model training, prediction) and frontend operations (form inputs, result visualizations, file uploads). The system is tested across all phases to identify and resolve potential errors.

TYPES OF TESTS

Unit Testing

Unit testing validates the correctness of individual modules, such as:

- Data preprocessing (missing value imputation, outlier handling)
- Model training functions (e.g., regression algorithms)
- Prediction logic (correct output for given inputs)

Each module was tested in isolation to ensure accurate outputs and adherence to expected behaviour. Libraries like unit test and pates were used for these validations.

Integration Testing

Integration testing focuses on verifying that components such as:

- Data input interface
- Preprocessing pipeline
- Model training workflow
- API-based and form-based predictions
- Visualization components

All interact as expected. This confirmed smooth data flow between frontend (HTML form inputs) and backend (Python ML models), validating real-world scenarios like uploading a dataset and obtaining predicted results.

Functional Testing

Functional tests were performed to verify system behaviour against the defined functional requirements:

- Valid Input: Correct numerical entries for wind speed, direction, etc., are processed accurately
- Output: Accurate power predictions are shown
- Systems/Procedures: Uploaded CSV is parsed and forwarded for processing correctly

System Testing

The complete integrated system was tested on a simulated deployment environment using actual datasets. System testing ensured:

- Entire workflow from data input to prediction is functional
- Trained models perform consistently on test data
- Pages (train.html, prediction.html) link correctly and respond quickly

White Box Testing

Testers examined internal logic such as model training routines, preprocessing code, and metric calculations to ensure proper implementation. They verified that:

- PCA properly reduces feature dimensions
- MICE handles missing values efficiently
- Correct correlation-based features are selected

Black Box Testing

From the user perspective, the system was tested without knowledge of the backend. This verified that:

- Input forms accept data smoothly
- Uploading datasets yields predictions
- Graphs match expected performance trends
- Users can navigate and receive results without errors

Test Strategy and Approach

- Manual field testing was performed on all pages
- Test cases included invalid and boundary data inputs
- Functionalities were tested against documented use-cases

Test Objectives

- Ensure all fields accept and validate correct input
- Ensure prediction page activates on correct conditions
- Ensure no system crashes or delays on valid data processing

Features to be Tested

- CSV upload functionality and field recognition
- Manual input prediction through the web form
- Prediction result accuracy and format
- Graph rendering and metric display
- Navigation between training and prediction modules

Integration Testing

Validated the interaction between:

- Data input components and preprocessing module
- Trained model and prediction handler
- Visualization and result display components

Result: All modules integrated successfully. No interface errors found.

Acceptance Testing

User Acceptance Testing was conducted with sample end-users (engineers or analysts) to confirm:

- Usability of the interface
- Clarity of results
- Navigation across system pages
- Understanding and interpretation of graphs

Results:

☒ All the test cases mentioned above passed successfully. No defects encountered. The system met all functional expectations and performance benchmarks.

8.1 TEST CASES:

S.No.	Test Case	Expected Result	Result	Remarks (If Fails)
1	User Registration	User registers successfully with unique email	Pass	Fails if email already exists
2	User Login	Valid credentials redirect to user dashboard	Pass	Fails for invalid credentials or unregistered users
3	Upload Dataset	CSV dataset is uploaded and validated	Pass	Fails if file format is invalid or required fields are missing
4	Train Model (KNN)	Model trains successfully with selected algorithm	Pass	Fails if data is missing or algorithm fails to initialize
5	Train Model (Linear Regression)	Linear Regression model trains and saves	Pass	Fails if training data is corrupted or incomplete
6	Train Model (Decision Tree)	Decision Tree model processes data and generates output	Pass	Fails if feature columns are mismatched
7	Predict Output (Manual Input)	Predicts wind power output from manual input	Pass	Fails if form is incomplete or contains invalid values
8	Predict Output (CSV Input)	Predicts power values for all rows in test CSV	Pass	Fails if CSV has inconsistent or missing data

9	View Prediction Results	Display of predicted power and comparison graphs	Pass	Fails if visualization libraries are not working
10	Accuracy Metrics (MAE, MSE, R ² , MAPE)	Evaluation metrics are calculated and shown	Pass	Fails if model was not trained or input dataset is empty
11	Dataset Preview	Displays uploaded dataset before model training	Pass	Fails if file is unreadable or headers are misaligned
12	Admin Login	Admin logs in and redirects to admin dashboard	Pass	Fails for incorrect credentials
13	Admin Activates User	Admin enables login for registered user	Pass	Fails if user record is not found
14	Navigation Between Pages (Train, Predict, Result)	Navigates smoothly between system pages	Pass	Fails if route configuration is broken
15	Output Visualization (Graphs, Charts)	Charts for actual vs predicted are generated	Pass	Fails if charting libraries like Plotly or Matplotlib are not integrated

CHAPTER 9

SAMPLE CODE

9. Sample Code

```
from django.conf import settings
from django.shortcuts import render, redirect
import pandas as pd

from users.forms import WindInputForm
from .models import UserRegistrationModel
from django.contrib import messages

def UserRegisterActions(request):
    if request.method == 'POST':
        user = UserRegistrationModel(
            name=request.POST['name'],
            loginid=request.POST['loginid'],
            password=request.POST['password'],
            mobile=request.POST['mobile'],
            email=request.POST['email'],
            locality=request.POST['locality'],
            address=request.POST['address'],
            city=request.POST['city'],
            state=request.POST['state'],
            status='waiting'
        )
        user.save()
        messages.success(request, "Registration successful!")
    return render(request, 'UserRegistrations.html')

def UserLoginCheck(request):
    if request.method == "POST":
        loginid = request.POST.get('loginid')
        pswd = request.POST.get('pswd')
        print("Login ID = ", loginid, ' Password = ', pswd)
```



```

try:
    check = UserRegistrationModel.objects.get(loginid=loginid, password=pswd)
    status = check.status
    print('Status is = ', status)
    if status == "activated":
        request.session['id'] = check.id
        request.session['loggeduser'] = check.name
        request.session['loginid'] = loginid
        request.session['email'] = check.email
        data = {'loginid': loginid}
        print("User id At", check.id, status)
        return render(request, 'users/UserHomePage.html', {})
    else:
        messages.success(request, 'Your Account Not at activated')
        return render(request, 'UserLogin.html')
except Exception as e:
    print('Exception is ', str(e))
    pass
    messages.success(request, 'Invalid Login id and password')
return render(request, 'UserLogin.html', {})

def UserHome(request):
    return render(request, 'users/UserHomePage.html', {})
def index(request):
    return render(request, "index.html")

def logout_view(request):
    return render(request, 'index.html')

import os
import io
import base64
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

```

```

import seaborn as sns
import joblib

from django.shortcuts import render
from django.conf import settings
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Move BASE_DIR definition to module level
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

def train_view(request):
    try:
        # Set paths
        dataset_path = os.path.join(BASE_DIR, 'media', 'wind_power_dataset_50000.csv')
        models_dir = os.path.join(BASE_DIR, 'models')
        os.makedirs(models_dir, exist_ok=True)

        # Load dataset
        df = pd.read_csv(dataset_path)
        print(f'Dataset loaded successfully with {len(df)} rows')

        # Drop datetime or non-numeric columns if any
        if 'Timestamp' in df.columns:
            df = df.drop(columns=['Timestamp'])

        # Ensure only numeric columns are used
        numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
        if "Active_Wind_Power_kW" not in numeric_cols:
            raise ValueError("Target column 'Active_Wind_Power_kW' not found in numeric columns.")
    
```

```

X = df.drop(columns=['Active_Wind_Power_kW'])
y = df['Active_Wind_Power_kW']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Model training
model = KNeighborsRegressor(n_neighbors=5)
model.fit(X_train_scaled, y_train)

# Prediction
y_pred = model.predict(X_test_scaled)

# Metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

# Save model and scaler
joblib.dump(model, os.path.join(models_dir, 'model.pkl'))
joblib.dump(scaler, os.path.join(models_dir, 'scaler.pkl'))

# Generate correlation matrix image
correlation_plot = None
try:
    corr = df.corr()
    plt.figure(figsize=(10, 8))
    sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", square=True)

```

```

plt.title("Correlation Matrix")

buf = io.BytesIO()
plt.savefig(buf, format='png')
buf.seek(0)
image_base64 = base64.b64encode(buf.read()).decode('utf-8')
buf.close()
plt.close()
correlation_plot = image_base64
except Exception as e:
    correlation_plot = None

# Context for template
context = {
    'mae': f'{mae:.2f}',
    'mse': f'{mse:.2f}',
    'rmse': f'{rmse:.2f}',
    'r2': f'{r2:.4f}',
    'correlation_plot': correlation_plot
}

return render(request, 'users/training.html', context)

except Exception as e:
    error_msg = f'Error during model training: {str(e)}'
    print(error_msg)
    return render(request, 'users/training.html', {'error': error_msg})

def predict_view(request):
    prediction = None

    if request.method == 'POST':
        # Load saved model and scaler
        model = joblib.load(os.path.join(BASE_DIR, 'models', 'model.pkl'))

```

```

scaler = joblib.load(os.path.join(BASE_DIR, 'models', 'scaler.pkl'))

# Get input data
try:
    features = [
        float(request.POST['wind_speed']),
        float(request.POST['wind_dir']),
        float(request.POST['temp']),
        float(request.POST['rotor_speed']),
        float(request.POST['pressure'])
    ]
    features_scaled = scaler.transform([features])
    prediction = model.predict(features_scaled)[0]
except Exception as e:
    prediction = f"Error: {e}"

return render(request, 'users/prediction.html', {'prediction': prediction})

def view_dataset(request):
    # Replace this with your real dataset logic
    return render(request, 'users/viewdataset.html')

import pandas as pd
from django.shortcuts import render
import os
from django.conf import settings

def view_dataset(request):
    dataset_path = os.path.join(settings.MEDIA_ROOT, 'wind_power_dataset_50000.csv') #
    Ensure file is there
    df = pd.read_csv(dataset_path)
    df = df.head(500) # Limit to first 500 rows
    table_html = df.to_html(classes='table table-striped table-bordered', index=False, border=0)
    return render(request, 'users/viewdataset.html', {'table': table_html})

```

CHAPTER 10

SCREENS SHORTS

10. SCREEN SHOTS

Evaluation of Machine Learning Algorithms for the Detection of Fake Bank Currency. Now-a-days all countries are facing major financial blow due to fake currency and to overcome from this problem author is employing various machine learning algorithms to predict whether currency is fake or not. As machine algorithms already proves it success in almost all fields such as Healthcare prediction, cyber-attack prediction, credit card fraud detection and many more. So author is suggesting to add machine learning power on fake currency detection.

Home page:

The Home Page serves as the entry point of the Wind Power Forecasting system. It provides an overview of the application, its purpose, and key features. This page typically includes navigation options such as User Registration, User Login, and Admin Login.

The main objective of the home page is to guide users to different modules of the system. It may also include a brief description of wind energy importance and how machine learning helps in forecasting wind power generation accurately.

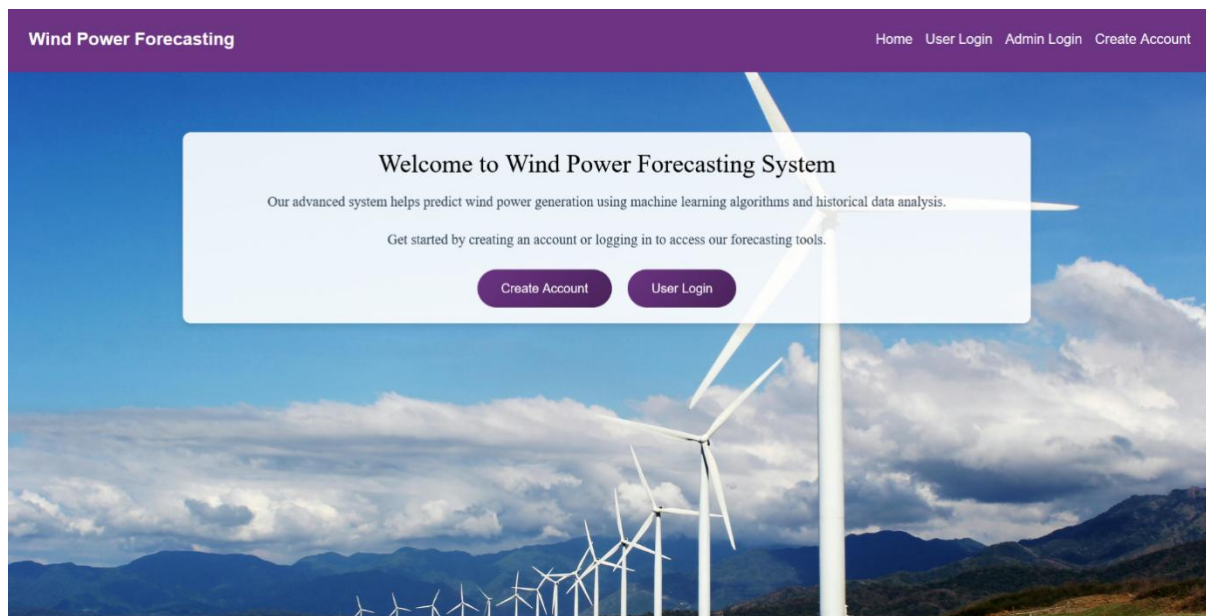


Fig no. 9.1 Home Page

User Registration Form:

The User Registration Form allows new users to create an account in the system. Users are required to provide basic details such as:

- Name
- Email ID
- Password
- Contact details

After successful registration, user data is stored securely in the database. This ensures that only authorized users can access the system features.

Validation is applied to ensure correct input format and prevent duplicate registrations.

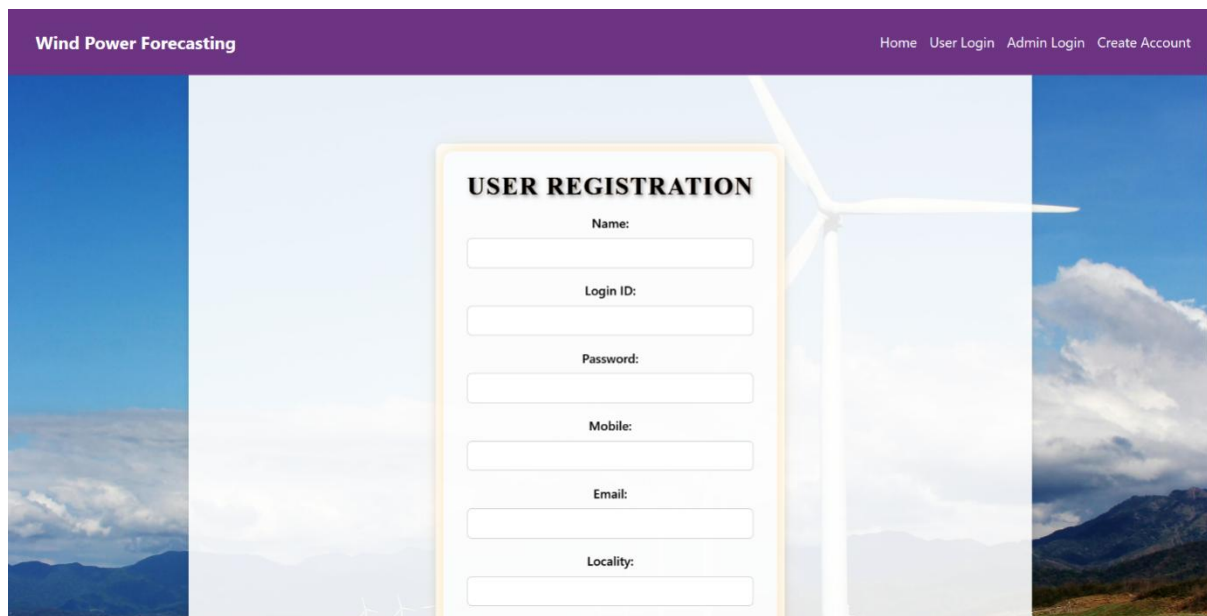
The screenshot shows a web application titled "Wind Power Forecasting" in a purple header. Navigation links include "Home", "User Login", "Admin Login", and "Create Account". The main content area features a "USER REGISTRATION" form with input fields for Name, Login ID, Password, Mobile, Email, and Locality. The background is a scenic image of a wind turbine in a mountainous area under a blue sky with clouds.

Fig no. 9.2 User Registration page

Admin Login Form:

The Admin Login Form is designed for system administrators. It requires valid credentials (username and password) to access the admin panel.

The admin has higher privileges compared to normal users, including:

- Managing users
- Viewing system data
- Monitoring predictions

Authentication ensures that only authorized personnel can access sensitive system functionalities.

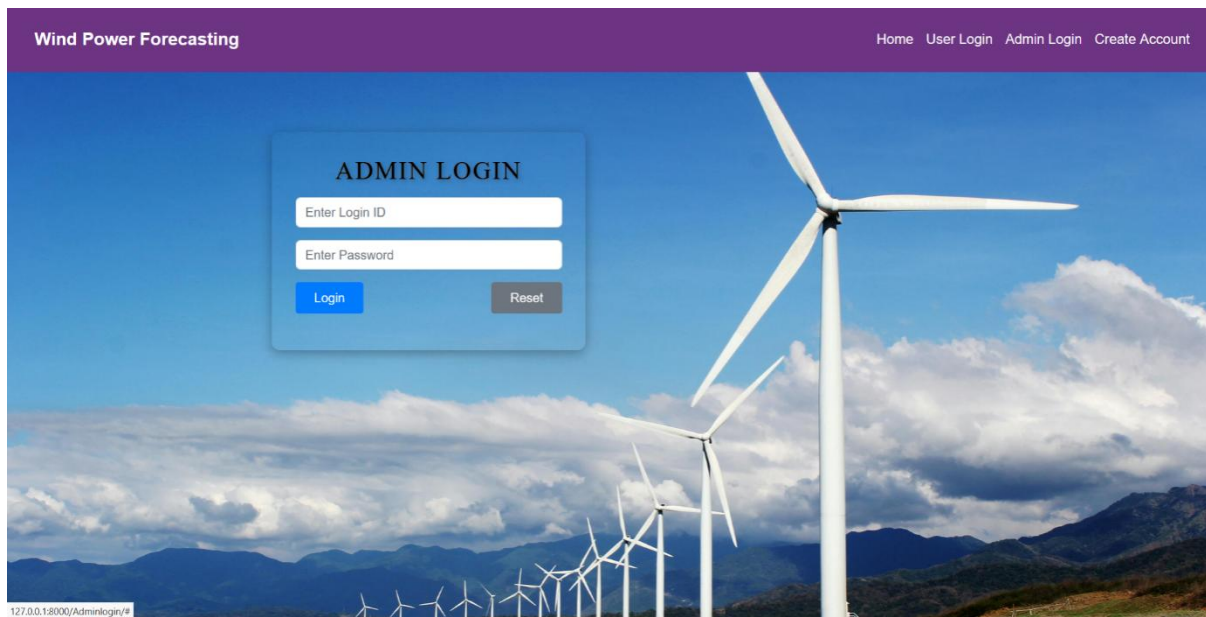


Fig no. 9.3 Admin Login Form

Admin Home:

The Admin Home Page acts as the control panel for the administrator. After login, the admin can perform various operations such as:

- Viewing registered users
- Activating or deactivating users
- Monitoring system performance

This page provides a centralized interface for managing the overall system efficiently

Activate Users:

This module allows the admin to approve or activate newly registered users. Until activation, users may not be able to log in.

Key functionalities include:

- Viewing list of pending users
- Activating authorized users

- Blocking suspicious users

This ensures system security and controlled access.

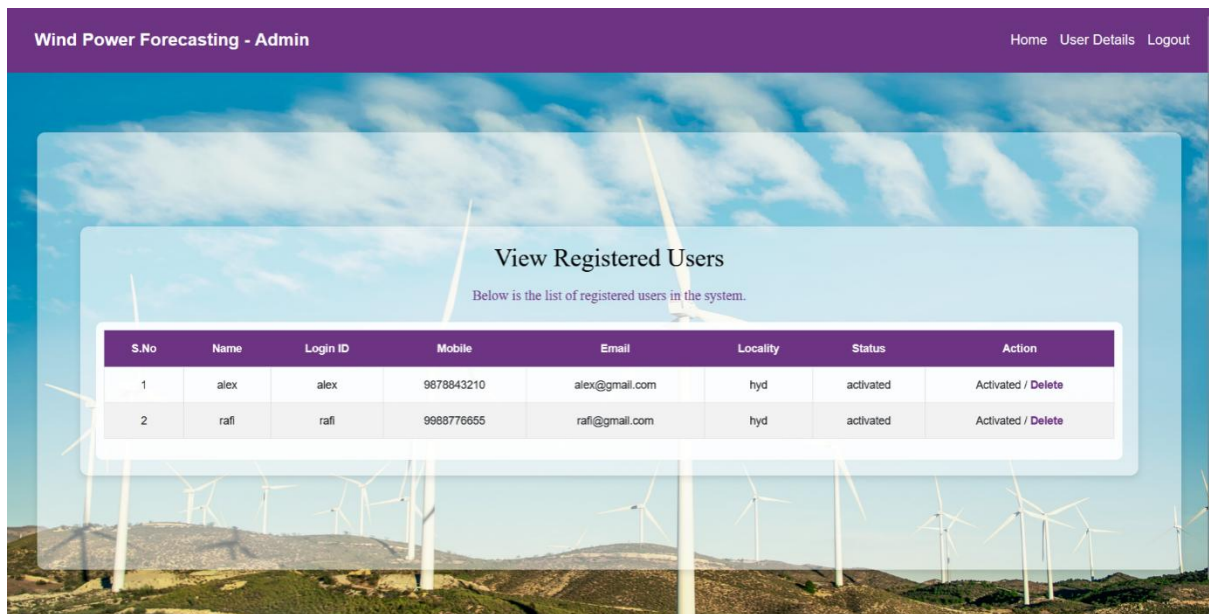


Fig no. 9.4 Activate Users

User Login Form:

This module allows the admin to approve or activate newly registered users. Until activation, users may not be able to log in.

Key functionalities include:

- Viewing list of pending users
- Activating authorized users
- Blocking suspicious users

This ensures system security and controlled access.

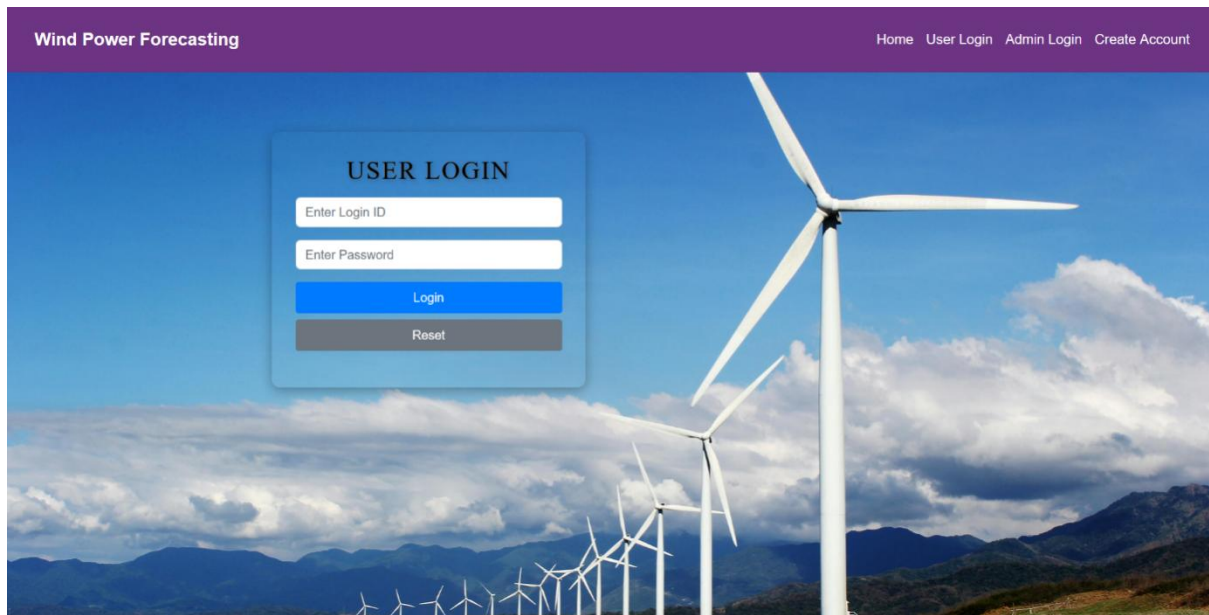


Fig no. 9.5 User Login Form

User Home Page:

The User Home Page is the dashboard for users after login. It provides access to all user functionalities such as:

- Viewing datasets
- Running machine learning
- Checking prediction results

This page acts as the main interface for interacting with the system.

Dataset View:

The Dataset View module allows users to explore the dataset used for wind power forecasting. It displays:

- Wind speed
- Temperature
- Pressure
- Power generation values

Users can analyse the dataset before applying machine learning models. This helps in understanding patterns and trends in wind energy data.

Wind Power Forecasting

HomeView DatasetTrainingPredictionLogout

Wind Power Dataset

Timestamp	Wind_Speed_mps	Wind_Direction_deg	Temperature_C	Rotor_Speed_rpm	Pressure_hPa	Active_Wind_Power_kW
2024-01-01 00:00:00	11.241785	0.997382	14.558421	24.759163	1027.957118	746.752114
2024-01-01 00:10:00	9.654339	137.708149	22.518025	17.694698	1014.033274	469.767665
2024-01-01 00:20:00	11.619221	329.057737	6.949929	18.922721	1015.622526	873.315780
2024-01-01 00:30:00	13.807575	341.191773	15.974644	24.086203	1027.813838	1363.358355
2024-01-01 00:40:00	9.414617	19.281329	10.237123	24.997443	1007.613567	463.153125
2024-01-01 00:50:00	9.414658	44.432089	17.114572	22.893568	1009.461860	502.142845
2024-01-01 01:00:00	13.948032	1.753324	5.584052	22.360612	1024.732189	1475.711413
2024-01-01 01:10:00	11.918587	312.722536	16.644641	19.111698	1005.820803	905.657806
2024-01-01 01:20:00	8.826314	25.519041	24.694511	16.574906	1007.946497	338.246566
2024-01-01 01:30:00	11.356400	38.844055	12.270885	18.594097	1005.800114	795.527633
2024-01-01 01:40:00	8.841456	81.044154	7.481937	20.085470	1005.181970	368.899435

Fig no. 9.6 View Data Set

ML Results:

This section displays the results of different machine learning algorithms applied to the dataset.

Common algorithms used:

- Linear Regression
- Random Forest
- Decision Tree

The results include:

- Accuracy
- Error metrics (MAE, RMSE)
- Comparison of models

This helps users identify the best-performing model for forecasting

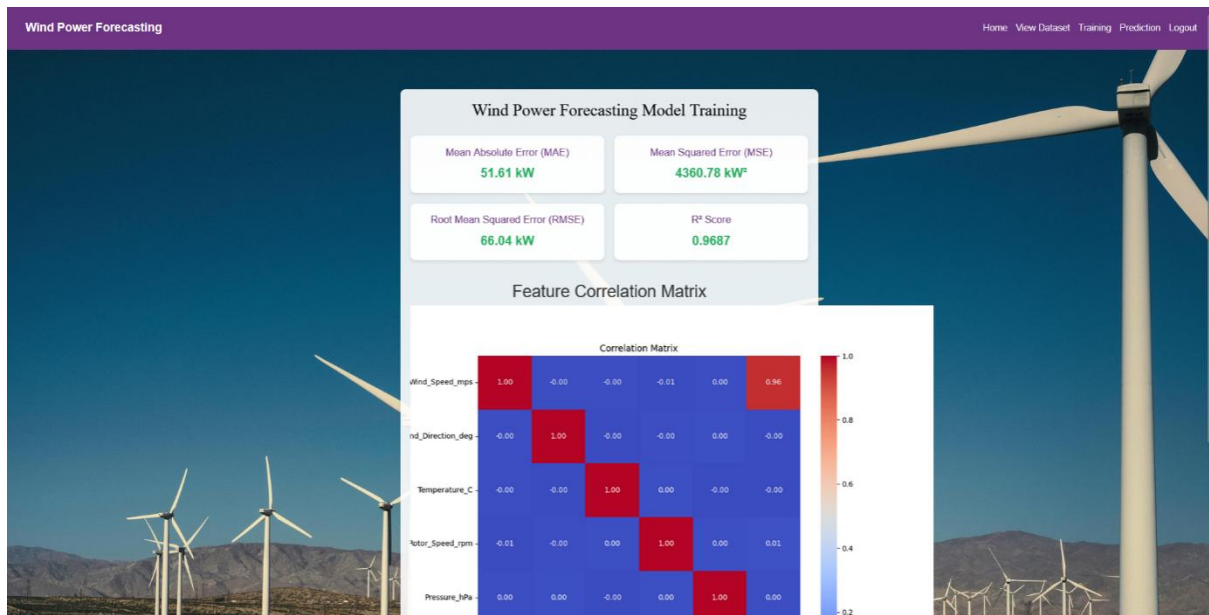


Fig no. 9.7 ML Results

Prediction page:

The Prediction Page allows users to input new data values such as:

- Wind speed
- Temperature

Based on the trained machine learning model, the system predicts the expected wind power output

The prediction form includes the following input fields:

- Wind Speed (m/s)
- Wind Direction (deg)
- Temperature (°C)
- Rotor Speed (rpm)
- Pressure (hPa)

A "Predict Wind Power" button is located at the bottom of the form.

Fig no. 9.8 Prediction Page

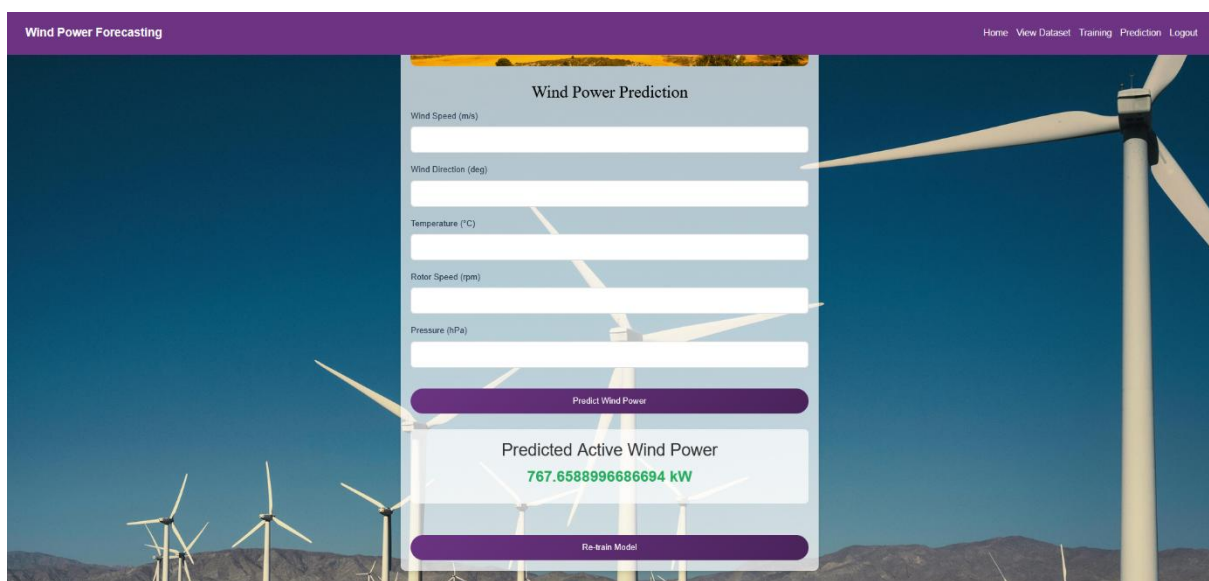
Prediction:

This section displays the final predicted wind power output.

Features include:

- Real-time prediction result
- Clear display of input vs output
- Graphical representation (optional)

This helps users understand how much power can be generated under given conditions.



The screenshot displays the 'Wind Power Forecasting' application interface. The background features a photograph of several wind turbines against a clear blue sky. A central white panel contains the 'Wind Power Prediction' form. The form includes input fields for 'Wind Speed (m/s)', 'Wind Direction (deg)', 'Temperature (°C)', 'Rotor Speed (rpm)', and 'Pressure (hPa)'. Below these fields is a purple button labeled 'Predict Wind Power'. The output section shows 'Predicted Active Wind Power' as '767.6588996686694 kW' in green text. At the bottom of the panel is a purple button labeled 'Re-train Model'. The top of the interface has a purple header bar with the title 'Wind Power Forecasting' and navigation links: 'Home', 'View Dataset', 'Training', 'Prediction', and 'Logout'.

Fig no. 9.9 Prediction Output

CHAPTER 11

CONCLUSION

11. CONCLUSION

This project presents a machine learning-based framework for forecasting active wind power using real-time turbine and environmental data. By evaluating multiple regression models—including Linear Regression, Ridge, Lasso, Decision Tree, Gradient Boosting, and K-Nearest Neighbours—on data collected at 10-minute intervals, the study demonstrates the effectiveness of data-driven techniques in improving wind energy predictability.

Through comprehensive preprocessing (including imputation, outlier handling, and dimensionality reduction via PCA) and rigorous model evaluation using MAE, MSE, R^2 , and MAPE metrics, the system identified the K-Nearest Neighbours Regressor as the most accurate and robust model. It consistently achieved high performance, outperforming other methods in terms of precision and generalization.

The findings highlight the feasibility of integrating such ML-based forecasting systems into renewable energy management infrastructures, thereby supporting more efficient energy planning, reduced grid instability, and better integration of wind power into existing grids. The scalable and cost-effective nature of the proposed system also positions it as a practical solution for real-world deployment in wind farms and smart grid applications.

12. REFERENCES

- [1] Chhotu Kumar and Anil Kumar Dudyala, —Banknote Authentication using Decision Tree rules and Machine Learning Techniques, International Conference on Advances in Computer Engineering and Applications (ICACEA), 2015.
- [2] Eugen Gillich and Volker Lohweg, —Banknote Authentication, 2014.
- [3] Thirunavukkarasu M, Dinakaran K, Satishkumar E.N and Gnanendra S, —Comparison of support vector machine(svm) and Back propagation network (bpn) methods in predicting the protein Virulence factors, Jr. of Industrial Pollution Control 33(2)(2017)pp 11-19.
- [4] Zan Huang, Hsinchun Chen, Chia-Jung-Hsu, Wun-Hwa Chen and Soushan Wuc, —Credit rating analysis with support vector machines and neural network: a market comparative study, 2004.
- [5] Ming-Chang Lee and Chang To, —Comparison of Support Vector Machine and Back Propagation Neural Network in Evaluating the Enterprise Financial Distress, International Journal of Artificial Intelligence & Applications 1.3 (2010) 31-43.
- [6] Prachi Damodhar Shahare and Ram Nivas Giri, —Comparative Analysis of Artificial Neural Network and Support Vector Machine Classification for Breast Cancer Detection, International Research Journal of Engineering and Technology, Dec-2015.
- [7] Fumiaki Takeda, Lalita Sakoobunthu and Hironobu Satou, —Thai Banknote Recognition Using Neural Network and Continues Learning by DSP Unit, International Conference on Knowledge-Based and Intelligent Information and Engineering Systems, 2003.
- [8] Costas Nastoulis, Apostolos Leros, and Nikolaos Bardis, —Banknote Recognition Based On Probabilistic Neural Network Models, Proceedings of the 10th WSEAS International Conference on SYSTEMS, Vouliagmeni, Athens, Greece, July 10- 12, 2006.
- [9] Sigeru Omatu, Michifumi Yoshioka and Yoshihisa Kosaka, —Bank Note Classification Using Neural Networks, IEEE, 2007.
- [10] Swati V. Walke and Prof. Dr. D. M. Chandwadkar, —Counterfeit Currency Recognition Using SVM With Note to Coin Exchanger, International Journal of Modern Trends in Engineering and Research, July 2015.
- [11] Sharmishta Desai, Shraddha Kabade, Apurva Bakshi, Apeksha Gunjal, Meghana Yeole, —Implementation of Multiple Kernel Support Vector Machine for Automatic Recognition and Classification of Counterfeit Notes, International Journal of Scientific & Engineering Research, October-2014.
- [12] Masato Aoba, Tetsuo Kikuchi, and Yoshiyasu Takefuji, —Euro Banknote Recognition System Using a Three layered Perceptron and RBF Networks, IPSJ Transactions on Mathematical

Modeling and it's Applications, May 2003.

- [13] <https://archive.ics.uci.edu/ml/datasets/banknote+authentication>.
- [14] https://www.researchgate.net/post/Where_must_we_use_variance_and_mean_of_image.
- [15] <http://www.itl.nist.gov/div898/handbook/eda/section3/eda35b.html>.
- [16] <http://www.astro.cornell.edu/research/projects/compression/entropy.html>.
- [17] Arti Patle and Deepak Singh Chouhan, —SVM Kernel Functions for Classification, ICATE 2013.
- [18] Chiara Gigliarano, Silvia Figini, Pietro Muliere, —Making classifier performance comparisons when ROC curves intersect, Computational Statistics and Data Analysis 77 (2014) 300–312.
- [19] https://en.wikipedia.org/wiki/Receiver_operating_characteristic.
- [20] Tushar Agasti, Gajanan Burand, Pratik Wade and P Chitra, —Fake currency detection using image processing, 14th ICSET-2017
- [21] Chinmay Bhurke, Meghana Sirdeshmukh, Prof. Mrs. M.S.Kanitkar, —Currency Recognition Using Image Processing, International Journal of Innovative Research in Computer and Communication Engineering, Vol. 3, Issue 5, May 2015